

Java AI 应用开发课程 自适应深讲版

参考流程图与大课讲义模式重新完善：不再固定每个知识点页数，而是按复杂度展开。复杂知识点给全流程图、组件拆解、工程骨架和生产治理；简单知识点短讲但讲透。

课程定位

Java 开发者转企业级 AI 应用开发

讲解方式

本质 - 组件 - 流程 - 示例 - 治理

页数策略

按知识点复杂度自适应，不机械凑页

重点变化

复杂点画全链路图，简单点讲清边界和应用

生成日期：2026-08-08

目录：D:\acme\AI-Learning\codex版

这版应该怎么学

学习 AI 应用开发，不是把每个名词平均展开，而是看它在系统中承担多大复杂度。像 Agent、RAG、Tool Calling、NL2SQL、治理这种复杂知识点，必须画全流程；像 temperature、向量维度、只读账号这种基础点，讲清边界和应用就够。

这版资料的阅读顺序，和《vibe coding.pdf》一样：先看真实问题，再看判断标准，最后才看流程图和代码骨架。

学习动作

1. 先看本质，确认这个知识点解决什么问题。
2. 再看组件，确认每个组件负责什么、不负责什么。
3. 复杂点一定自己重画流程图，尤其是 Agent、RAG、Tool、NL2SQL。
4. 最后用生产坑和验收指标倒推：如果上线，怎么知道它是好的？

掌握标准

- 能用自己的话讲清输入、输出、边界、失败路径。
- 能把复杂知识点画成流程图或泳道图。
- 能写出最小 Java 工程骨架。
- 能说出质量、成本、安全、权限和审计指标。

AI 应用开发认知 模块架构图

建立企业级 AI 应用的总视角：模型不是系统本身，AI 应用是模型、上下文、知识、工具、流程和治理组合出来的工程系统。

这一章先看什么： AI 应用架构 不是背概念，而是看它在系统里怎么落位。先把 先选模型再找场景；只画组件不画数据流 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

LLM 与传统后端的差异 核心讲解

本质：LLM 是概率生成系统，传统后端是确定逻辑系统。企业 AI 的核心工作，是在不确定能力外面包一层确定工程边界。

定位：LLM 与传统后端的差异 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

LLM 与传统后端的差异 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



关键误区：最常见的误区是把 AI 架构画成“前端 -> 模型 -> 答案”。真实企业链路至少还要有知识、工具、权限、审计、评估、降级和反馈，否则 Demo 越漂亮，上线越危险。

判断是否学会：你能讲清「LLM 与传统后端的差异」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

LLM 与传统后端的差异 组件拆解

1. 组件职责

1. 业务价值：回答“为什么要做”。学习「LLM 与传统后端的差异」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 治理闭环：负责让「LLM 与传统后端的差异」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

LLM 与传统后端的差异 工程示例

1. 最小工程骨架

```
// LLM与传统后端的差异: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「LLM 与传统后端的差异」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

LLM 与传统后端的差异 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「LLM 与传统后端的差异」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「LLM 与传统后端的差异」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「LLM 与传统后端的差异」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「LLM 与传统后端的差异」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- LLM 与传统后端的差异 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 LLM 与传统后端的差异 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「LLM 与传统后端的差异」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

AI 应用六层架构 核心讲解

本质：企业 AI 至少包含模型层、上下文层、知识层、工具层、编排层和治理层。缺任何一层，Demo 上线后都会暴露问题。

定位：AI 应用六层架构 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

AI 应用六层架构 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



关键误区：最常见的误区是把 AI 架构画成“前端 -> 模型 -> 答案”。真实企业链路至少还要有知识、工具、权限、审计、评估、降级和反馈，否则 Demo 越漂亮，上线越危险。

判断是否学会：你能讲清「AI 应用六层架构」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

AI 应用六层架构 组件拆解

1. 组件职责

1. 业务价值：回答“为什么要做”。学习「AI 应用六层架构」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 治理闭环：负责让「AI 应用六层架构」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

AI 应用六层架构 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

AI 应用六层架构 工程示例

1. 最小工程骨架

```
// AI应用六层架构: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「AI 应用六层架构」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

AI 应用六层架构 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「AI 应用六层架构」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「AI 应用六层架构」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「AI 应用六层架构」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「AI 应用六层架构」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- AI 应用六层架构 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 AI 应用六层架构 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「AI 应用六层架构」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Token 与上下文窗口 核心讲解

本质：模型看到的是 token 序列，不是无限记忆。上下文窗口决定模型一次能参考多少信息，也直接影响成本和稳定性。

定位：Token 与上下文窗口 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Token 与上下文窗口 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



Token 与上下文窗口 组件拆解

1. 组件职责

1. 业务价值：回答“为什么要做”。学习「Token 与上下文窗口」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 治理闭环：负责让「Token 与上下文窗口」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Token 与上下文窗口 工程示例

1. 最小工程骨架

```
// Token与上下文窗口: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Token 与上下文窗口」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Token 与上下文窗口 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「Token 与上下文窗口」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「Token 与上下文窗口」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「Token 与上下文窗口」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「Token 与上下文窗口」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- Token 与上下文窗口的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Token 与上下文窗口失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Token 与上下文窗口」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

模型能力边界 核心讲解

本质：模型擅长语言理解、生成和归纳，但不天然具备企业实时事实、权限判断和业务执行能力。

定位：模型能力边界 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

模型能力边界 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



模型能力边界 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「模型能力边界」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「模型能力边界」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

模型能力边界 工程示例

1. 最小工程骨架

```
// 模型能力边界: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「模型能力边界」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

模型能力边界 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「模型能力边界」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「模型能力边界」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「模型能力边界」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「模型能力边界」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 模型能力边界 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 模型能力边界 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「模型能力边界」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

幻觉的工程本质 核心讲解

本质：幻觉不是简单 bug，而是资料不足、约束不足或目标不清时，模型仍倾向生成合理文本。工程上要用 RAG、工具、拒答和评估约束。

定位：幻觉的工程本质 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

幻觉的工程本质 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



幻觉的工程本质 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「幻觉的工程本质」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「幻觉的工程本质」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

幻觉的工程本质 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

幻觉的工程本质 工程示例

1. 最小工程骨架

```
// 幻觉的工程本质: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「幻觉的工程本质」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

幻觉的工程本质 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「幻觉的工程本质」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「幻觉的工程本质」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「幻觉的工程本质」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「幻觉的工程本质」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 幻觉的工程本质 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 幻觉的工程本质 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「幻觉的工程本质」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

AI 应用价值判断 核心讲解

本质：不是所有功能都值得 AI 化。适合 AI 的任务通常语言密集、规则复杂、信息分散、人工成本高。

定位：AI 应用价值判断 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

AI 应用价值判断 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



AI 应用价值判断 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「AI 应用价值判断」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「AI 应用价值判断」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「AI 应用价值判断」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「AI 应用价值判断」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- AI 应用价值判断 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 AI 应用价值判断 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「AI 应用价值判断」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Java 开发者的优势 核心讲解

本质：Java 开发者擅长系统边界、事务、权限、稳定性和工程化，这正是企业 AI 落地最缺的能力。

定位：Java 开发者的优势 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Java 开发者的优势 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

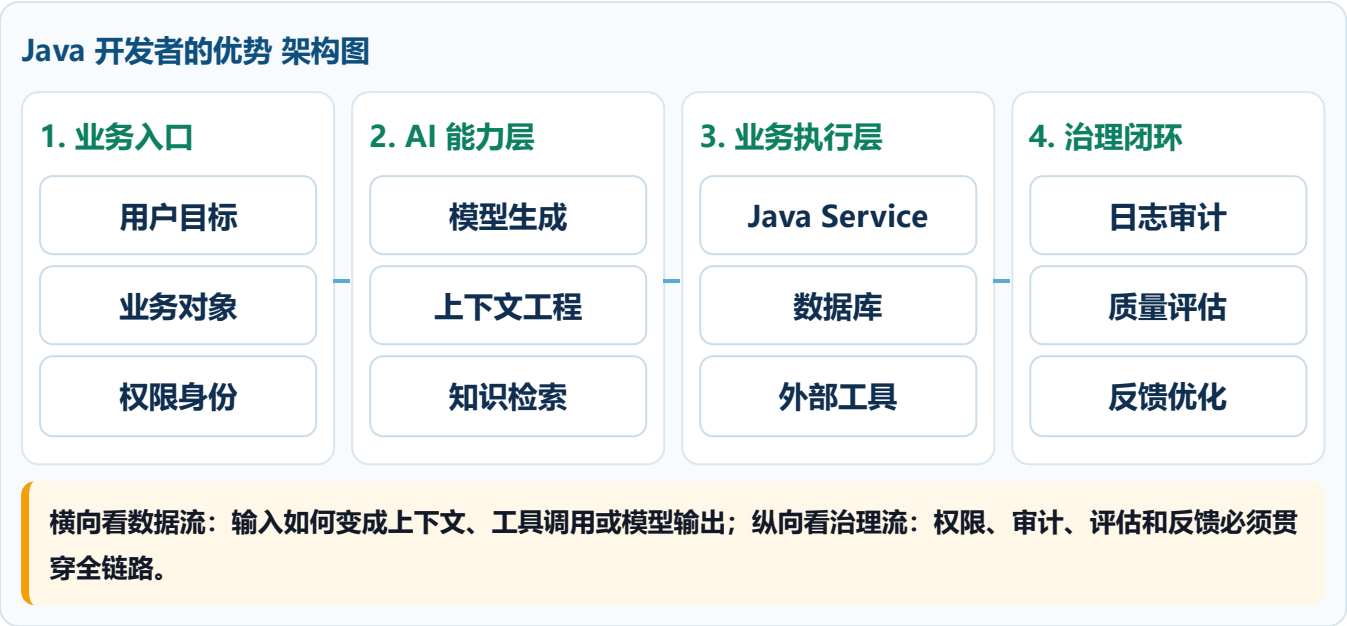
1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



关键误区：最常见的误区是把 AI 架构画成“前端 -> 模型 -> 答案”。真实企业链路至少还要有知识、工具、权限、审计、评估、降级和反馈，否则 Demo 越漂亮，上线越危险。

判断是否学会：你能讲清「Java 开发者的优势」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Java 开发者的优势 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「Java 开发者的优势」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「Java 开发者的优势」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「Java 开发者的优势」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「Java 开发者的优势」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- Java 开发者的优势 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Java 开发者的优势 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Java 开发者的优势」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

从聊天到 workflow 核心讲解

本质：聊天只是入口，真正的企业价值来自把回答变成流程节点、数据更新、审批建议和可追踪结论。

定位：从聊天到 workflow 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

从聊天到 workflow 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



关键误区：最常见的误区是把 AI 架构画成“前端 -> 模型 -> 答案”。真实企业链路至少还要有知识、工具、权限、审计、评估、降级和反馈，否则 Demo 越漂亮，上线越危险。

判断是否学会：你能讲清「从聊天到 workflow」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

从聊天到工作流 组件拆解

1. 组件职责

1. 业务价值：回答“为什么要做”。学习「从聊天到工作流」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 治理闭环：负责让「从聊天到工作流」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

从聊天到工作流 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

从聊天到 workflows 工程示例

1. 最小工程骨架

```
// 从聊天到 workflows: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「从聊天到 workflows」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

从聊天到 workflow 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「从聊天到 workflow」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「从聊天到 workflow」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「从聊天到 workflow」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「从聊天到 workflow」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 从聊天到 workflow 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果从聊天到 workflow 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「从聊天到 workflow」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

人机协作闭环 核心讲解

本质：AI 负责分析、生成和建议，人负责目标、判断、确认和反馈，系统负责记录过程并推动迭代。

定位：人机协作闭环 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

人机协作闭环 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



人机协作闭环 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「人机协作闭环」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「人机协作闭环」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

人机协作闭环 工程示例

1. 最小工程骨架

```
// 人机协作闭环: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「人机协作闭环」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

人机协作闭环 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「人机协作闭环」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「人机协作闭环」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「人机协作闭环」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「人机协作闭环」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 人机协作闭环 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 人机协作闭环 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「人机协作闭环」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

学习路线总图 核心讲解

本质：学习要按模型调用、Prompt、Embedding、RAG、Tool、Agent、治理逐层推进，不要一开始就追复杂 Agent。

定位：学习路线总图 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

学习路线总图 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



学习路线总图 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「学习路线总图」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「学习路线总图」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「学习路线总图」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「学习路线总图」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

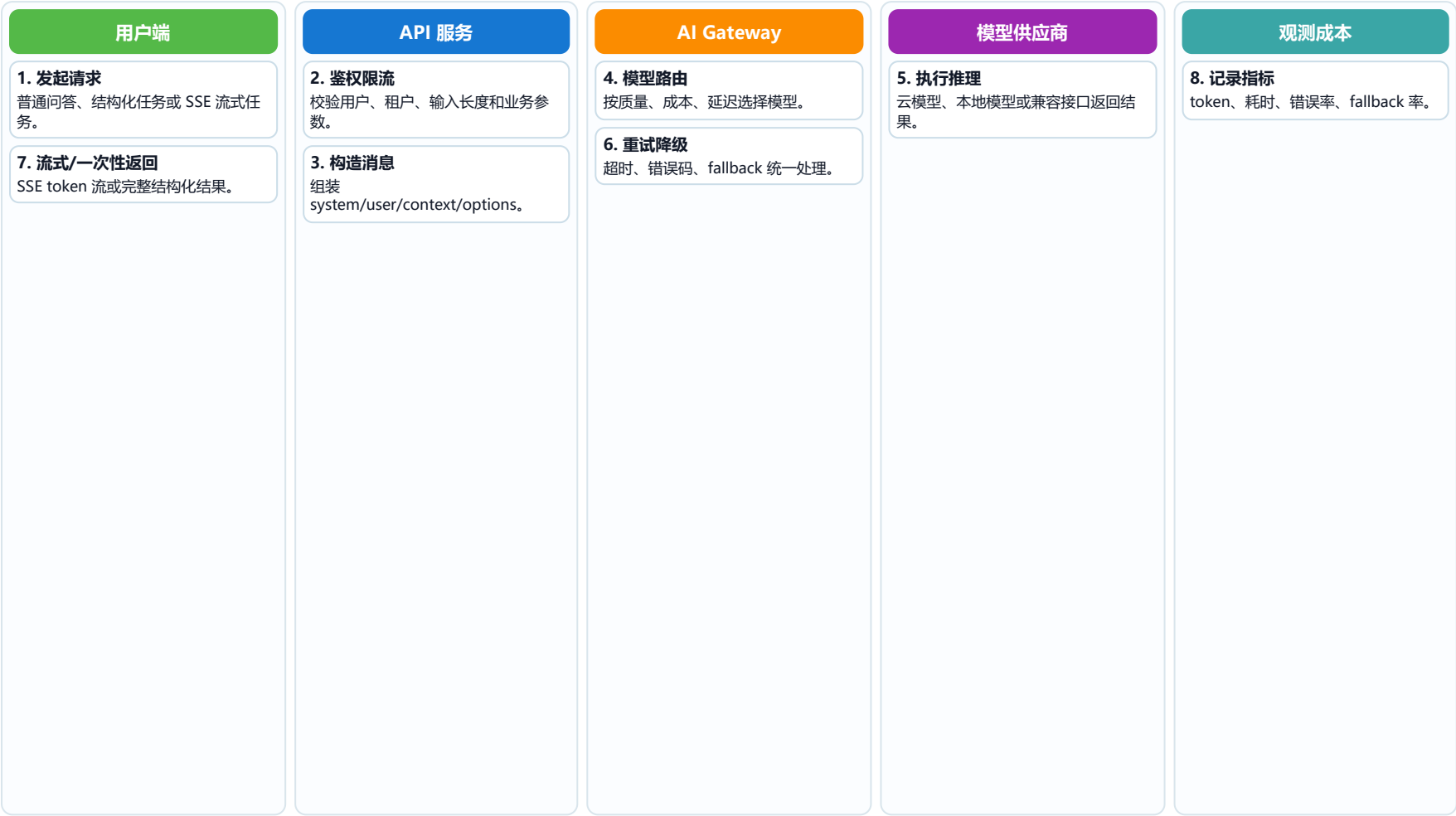
- 学习路线总图 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 学习路线总图 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「学习路线总图」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

模型 API 与流式聊天 模块架构图

把模型调用封装成稳定后端能力，重点是路由、流式、超时、fallback、成本和调用审计。

这一章先看什么：模型 API 工程化 不是背概念，而是看它在系统里怎么落位。先把 Controller 直接调模型；没有超时和降级 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Chat Completion 调用链路 核心讲解

本质：一次模型调用包含 system/user 消息、参数、上下文、模型选择、响应解析和错误处理，不只是一个 HTTP 请求。

定位：Chat Completion 调用链路 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Chat Completion 调用链路 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



Chat Completion 调用链路 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「Chat Completion 调用链路」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「Chat Completion 调用链路」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「Chat Completion 调用链路」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「Chat Completion 调用链路」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Chat Completion 调用链路 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Chat Completion 调用链路 工程示例

1. 最小工程骨架

```
// ChatCompletion调用链路: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Chat Completion 调用链路」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Chat Completion 调用链路 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「Chat Completion 调用链路」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「Chat Completion 调用链路」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「Chat Completion 调用链路」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「Chat Completion 调用链路」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- Chat Completion 调用链路 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Chat Completion 调用链路 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Chat Completion 调用链路」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

System Prompt 与 User Prompt 核心讲解

本质：System Prompt 定义角色和边界，User Prompt 承载具体任务。两者混用会让提示词不可治理。

定位：System Prompt 与 User Prompt 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

System Prompt 与 User Prompt 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

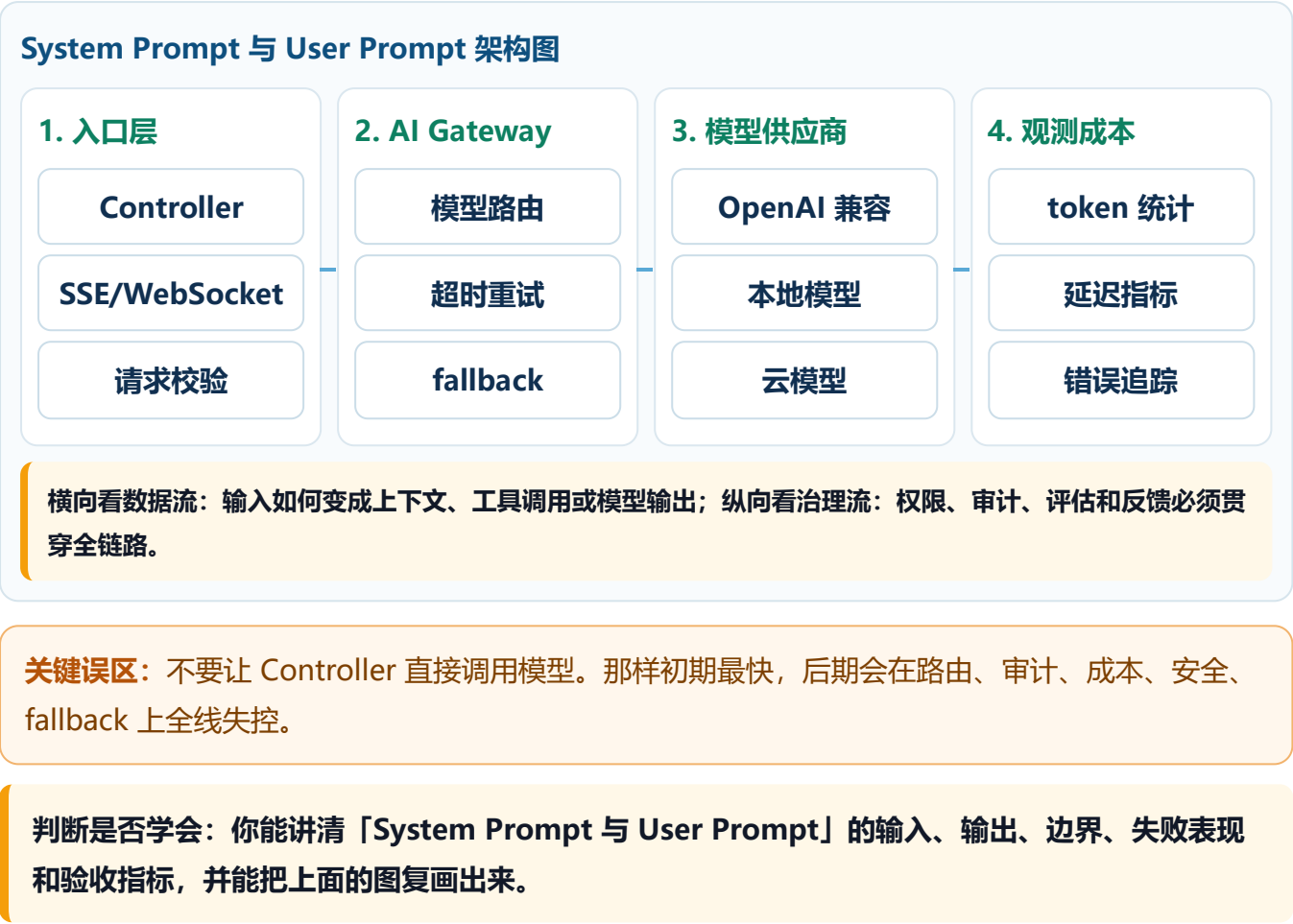
1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



System Prompt 与 User Prompt 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「System Prompt 与 User Prompt」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「System Prompt 与 User Prompt」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「System Prompt 与 User Prompt」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「System Prompt 与 User Prompt」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

System Prompt 与 User Prompt 工程示例

1. 最小工程骨架

```
// SystemPrompt与UserPrompt: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「System Prompt 与 User Prompt」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

System Prompt 与 User Prompt 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「System Prompt 与 User Prompt」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「System Prompt 与 User Prompt」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「System Prompt 与 User Prompt」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「System Prompt 与 User Prompt」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- System Prompt 与 User Prompt 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 System Prompt 与 User Prompt 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「System Prompt 与 User Prompt」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

模型参数 temperature/top_p 核心讲解

本质：参数决定输出稳定性和发散度。事实问答偏低温度，创意生成可提高。

定位：模型参数 temperature/top_p 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

模型参数 temperature/top_p 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

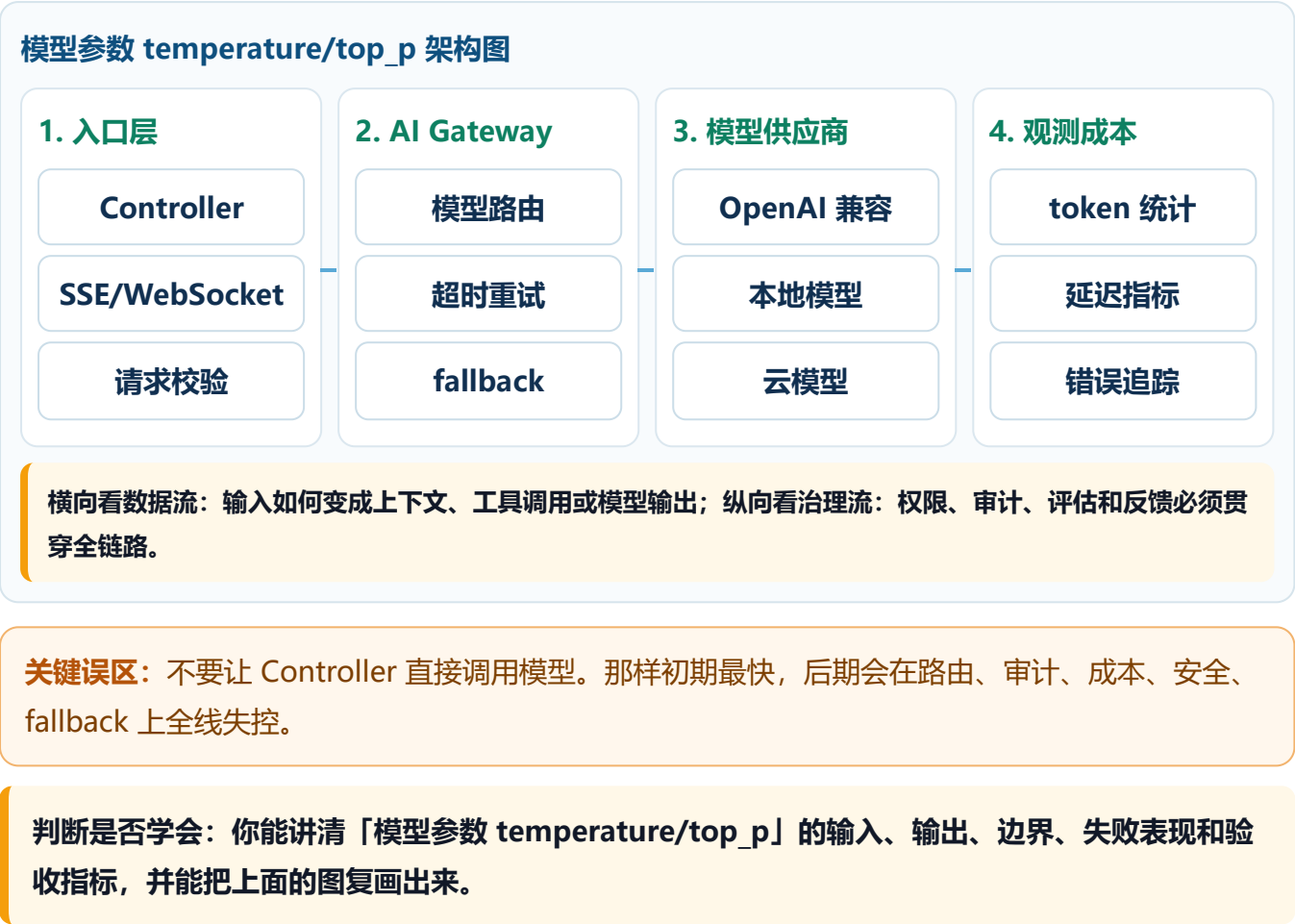
1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



模型参数 temperature/top_p 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「模型参数 temperature/top_p」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「模型参数 temperature/top_p」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「模型参数 temperature/top_p」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「模型参数 temperature/top_p」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- 模型参数 temperature/top_p 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 模型参数 temperature/top_p 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「模型参数 temperature/top_p」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

SSE 流式输出 核心讲解

本质：流式输出优化用户感知等待，但带来断线、取消、异常补偿、审计和前端状态管理复杂度。

定位：SSE 流式输出 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

SSE 流式输出 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



SSE 流式输出 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「SSE 流式输出」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「SSE 流式输出」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「SSE 流式输出」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「SSE 流式输出」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

SSE 流式输出 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

SSE 流式输出 工程示例

1. 最小工程骨架

```
// SSE流式输出: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「SSE 流式输出」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

SSE 流式输出 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「SSE 流式输出」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「SSE 流式输出」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「SSE 流式输出」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「SSE 流式输出」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- SSE 流式输出 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 SSE 流式输出 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「SSE 流式输出」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

模型路由策略 核心讲解

本质：不同模型适合不同任务。企业要按质量、成本、延迟、隐私和可用性做路由。

定位：模型路由策略 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

模型路由策略 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



模型路由策略 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「模型路由策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「模型路由策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「模型路由策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「模型路由策略」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

模型路由策略 工程示例

1. 最小工程骨架

```
// 模型路由策略: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「模型路由策略」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

模型路由策略 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「模型路由策略」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「模型路由策略」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「模型路由策略」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「模型路由策略」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- 模型路由策略 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 模型路由策略 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「模型路由策略」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

超时、重试与 fallback 核心讲解

本质：模型服务不稳定是常态。系统必须有超时、有限重试、降级模型和友好错误。

定位：超时、重试与 fallback 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

超时、重试与 fallback 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



超时、重试与 fallback 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「超时、重试与 fallback」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「超时、重试与 fallback」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「超时、重试与 fallback」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「超时、重试与 fallback」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

超时、重试与 fallback 工程示例

1. 最小工程骨架

```
// 超时、重试与fallback: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「超时、重试与 fallback」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

超时、重试与 fallback 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「超时、重试与 fallback」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「超时、重试与 fallback」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「超时、重试与 fallback」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「超时、重试与 fallback」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- 超时、重试与 fallback 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 超时、重试与 fallback 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「超时、重试与 fallback」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Token 成本核算 核心讲解

本质：成本由输入 token、输出 token、模型价格、重试、上下文长度和缓存命中共同决定。

定位：Token 成本核算 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Token 成本核算 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



Token 成本核算 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「Token 成本核算」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「Token 成本核算」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「Token 成本核算」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「Token 成本核算」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Token 成本核算 工程示例

1. 最小工程骨架

```
// Token成本核算: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Token 成本核算」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Token 成本核算 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「Token 成本核算」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「Token 成本核算」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「Token 成本核算」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「Token 成本核算」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- Token 成本核算 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Token 成本核算 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Token 成本核算」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt Cache 核心讲解

本质：重复问题或稳定上下文可以缓存，但必须注意权限、时效和个性化差异。

定位：Prompt Cache 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt Cache 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



Prompt Cache 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「Prompt Cache」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「Prompt Cache」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「Prompt Cache」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「Prompt Cache」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- Prompt Cache 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt Cache 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt Cache」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

OpenAI 兼容接口 核心讲解

本质：兼容接口不代表行为完全一致，参数、流式、错误码和工具调用都要测试。

定位：OpenAI 兼容接口 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

OpenAI 兼容接口 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

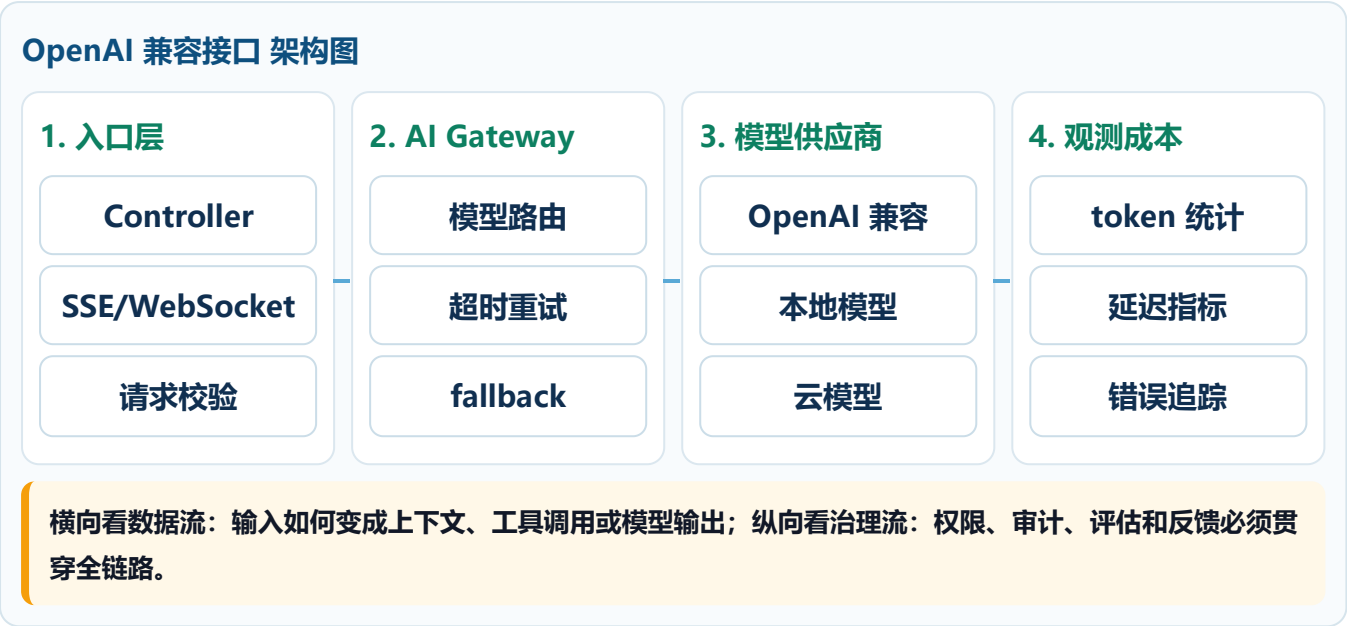
1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



关键误区：不要让 Controller 直接调用模型。那样初期最快，后期会在路由、审计、成本、安全、fallback 上全线失控。

判断是否学会：你能讲清「OpenAI 兼容接口」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

OpenAI 兼容接口 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「OpenAI 兼容接口」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「OpenAI 兼容接口」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「OpenAI 兼容接口」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「OpenAI 兼容接口」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

- OpenAI 兼容接口 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 OpenAI 兼容接口 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「OpenAI 兼容接口」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

模型调用日志 核心讲解

本质：日志是评估、成本、审计和安全的基础，不能只为排错记录。

定位：模型调用日志 属于「模型 API 工程化」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

模型调用日志 的本质是把不稳定的外部模型服务包装成稳定的企业后端能力。模型 API 不只是 HTTP 调用，它同时牵涉消息协议、参数、流式传输、路由、成本、重试、fallback 和日志。

1. 运行机制拆解

1. 一次调用先由业务请求生成 messages：system 定边界，user 给任务，assistant 保留历史，tool/result 补充外部事实。消息结构混乱，模型行为就会漂。
2. 模型参数不是玄学调参：temperature 控制随机性，max tokens 控制输出上限，stop/response format/tools 决定能不能被系统稳定消费。
3. AI Gateway 要负责模型路由、超时、重试、降级、限流和成本统计；业务代码不应该到处散落 provider SDK。
4. SSE 流式输出提升体验，但服务端必须处理客户端断开、任务取消、异常事件、审计补记和前端重连状态。

2. Java 工程落地抓手

- 定义统一 ModelClient 接口，屏蔽 OpenAI 兼容、国产模型、本地模型的差异。
- 每次调用生成 requestId，记录模型、Prompt 版本、input/output token、耗时、状态码、重试次数和降级原因。
- 事实问答类任务默认低温度、短超时、有限重试；创意生成可以更高温度，但必须限制输出长度和预算。

3. 核心架构图



模型调用日志 组件拆解

1. 组件职责

1. 消息结构：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 模型参数：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 模型网关：这是「模型调用日志」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. SSE 流式通道：这是「模型调用日志」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 重试/fallback：这是「模型调用日志」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
6. token 成本日志：负责让「模型调用日志」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

模型调用日志 工程示例

1. 最小工程骨架

```
// 模型调用日志: model gateway
Route route = router.choose(taskType, quality, cost, latency);
AiResponse response = modelClient.call(messages, route.options(), timeout);
Fallback fallback = fallbackPolicy.onFailure(response);
metrics.record(model, tokens, latency, status);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「模型调用日志」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

模型调用日志 生产坑与验收

1. 常见生产坑

1. Controller 直接调模型：对「模型调用日志」来说，处理建议是：抽出统一 AiGateway，Controller 不直接调模型。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有超时和降级：对「模型调用日志」来说，处理建议是：设置连接超时、读取超时、最大重试次数和降级模型。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 token 成本统计：对「模型调用日志」来说，处理建议是：记录 input/output token、模型单价、重试次数、调用人和业务线。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 流式输出缺少取消和异常处理：对「模型调用日志」来说，处理建议是：SSE 增加断线取消、心跳、异常事件、前端重连提示和服务端审计补记。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- P95 延迟
- 首 token 时间
- fallback 率
- token 成本
- 错误率

3. 学完必须能回答

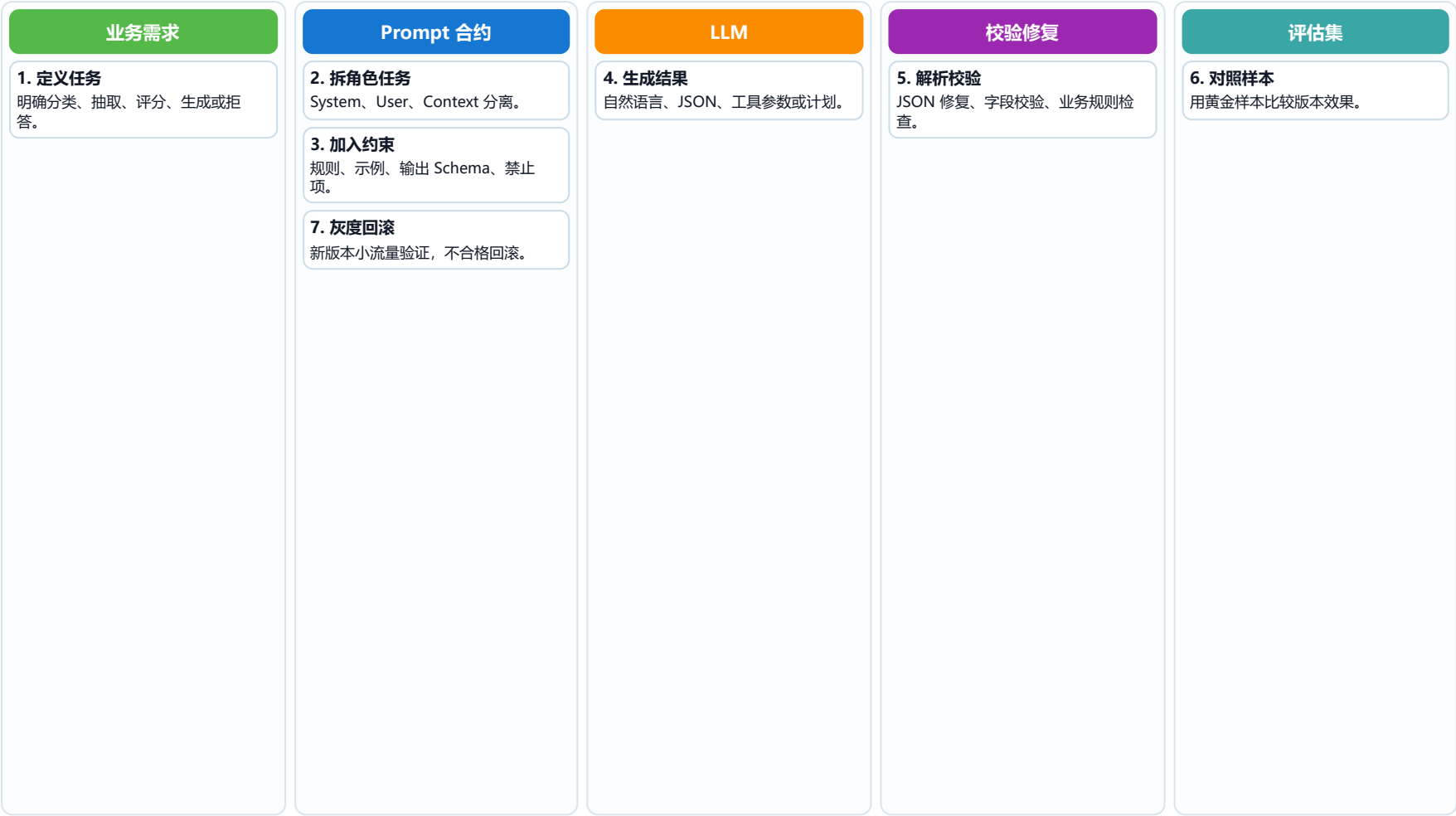
- 模型调用日志 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 模型调用日志 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「模型调用日志」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt 与结构化输出 模块架构图

把 Prompt 当成接口协议，让模型输出能稳定进入 Java 系统。

这一章先看什么： Prompt 与结构化输出 不是背概念，而是看它在系统里怎么落位。先把 把提示词写成大作文；只约束格式不约束业务 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Prompt 是接口协议 核心讲解

本质： Prompt 的作用是定义模型执行任务的协议，不是写一段求模型听话的作文。

定位： Prompt 是接口协议 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt 是接口协议 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「Prompt 是接口协议」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Prompt 是接口协议 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「Prompt 是接口协议」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt 是接口协议 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Prompt 是接口协议 工程示例

1. 最小工程骨架

```
// Prompt是接口协议: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt 是接口协议」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Prompt 是接口协议 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「Prompt 是接口协议」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「Prompt 是接口协议」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「Prompt 是接口协议」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「Prompt 是接口协议」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- Prompt 是接口协议 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt 是接口协议 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt 是接口协议」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

角色与任务拆分 核心讲解

本质：角色定义模型身份，任务定义要完成的工作，拆开后才可复用、测试和治理。

定位：角色与任务拆分 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

角色与任务拆分 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「角色与任务拆分」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

角色与任务拆分 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「角色与任务拆分」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

角色与任务拆分 工程示例

1. 最小工程骨架

```
// 角色与任务拆分: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「角色与任务拆分」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

角色与任务拆分 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「角色与任务拆分」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「角色与任务拆分」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「角色与任务拆分」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「角色与任务拆分」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- 角色与任务拆分 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 角色与任务拆分 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「角色与任务拆分」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

上下文选择 核心讲解

本质：上下文围绕任务目标选择，不是越多越好。多余上下文会增加成本并引入干扰。

定位：上下文选择 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

上下文选择 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



上下文选择 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「上下文选择」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

上下文选择 工程示例

1. 最小工程骨架

```
// 上下文选择: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「上下文选择」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

上下文选择 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「上下文选择」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「上下文选择」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「上下文选择」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「上下文选择」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- 上下文选择 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 上下文选择 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「上下文选择」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Few-shot 示例 核心讲解

本质：少量高质量示例能约束输出风格和判断尺度，但示例必须覆盖边界情况。

定位：Few-shot 示例 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Few-shot 示例 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「Few-shot 示例」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Few-shot 示例 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「Few-shot 示例」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Few-shot 示例 工程示例

1. 最小工程骨架

```
// Few-shot示例: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Few-shot 示例」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Few-shot 示例 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「Few-shot 示例」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「Few-shot 示例」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「Few-shot 示例」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「Few-shot 示例」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- Few-shot 示例 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Few-shot 示例 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Few-shot 示例」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

结构化输出 核心讲解

本质：结构化输出让 AI 结果进入业务系统，必须配合 JSON Schema、Java record、校验和异常处理。

定位：结构化输出 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

结构化输出 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「结构化输出」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

结构化输出 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「结构化输出」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

结构化输出 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

结构化输出 工程示例

1. 最小工程骨架

```
// 结构化输出: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「结构化输出」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

结构化输出 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「结构化输出」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「结构化输出」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「结构化输出」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「结构化输出」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- 结构化输出 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 结构化输出 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「结构化输出」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

JSON 修复策略 核心讲解

本质：模型偶尔会输出非法 JSON，系统需要提取、修复、重试和降级。

定位：JSON 修复策略 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

JSON 修复策略 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「JSON 修复策略」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

JSON 修复策略 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「JSON 修复策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

JSON 修复策略 工程示例

1. 最小工程骨架

```
// JSON修复策略: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「JSON 修复策略」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

JSON 修复策略 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「JSON 修复策略」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「JSON 修复策略」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「JSON 修复策略」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「JSON 修复策略」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- JSON 修复策略 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 JSON 修复策略 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「JSON 修复策略」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

输出字段设计 核心讲解

本质：字段越贴近业务决策，AI 越有价值。字段要区分结论、证据、风险、缺失信息和建议动作。

定位：输出字段设计 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

输出字段设计 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



输出字段设计 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「输出字段设计」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

输出字段设计 工程示例

1. 最小工程骨架

```
// 输出字段设计: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「输出字段设计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

输出字段设计 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「输出字段设计」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「输出字段设计」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「输出字段设计」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「输出字段设计」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- 输出字段设计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 输出字段设计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「输出字段设计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt Injection 核心讲解

本质：用户可能通过输入诱导模型忽略规则。防护要结合系统提示词、工具权限和输出校验。

定位：Prompt Injection 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt Injection 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「Prompt Injection」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Prompt Injection 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「Prompt Injection」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt Injection 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Prompt Injection 工程示例

1. 最小工程骨架

```
// PromptInjection: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt Injection」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Prompt Injection 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「Prompt Injection」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「Prompt Injection」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「Prompt Injection」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「Prompt Injection」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- Prompt Injection 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt Injection 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt Injection」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt 版本管理 核心讲解

本质： Prompt 是线上逻辑，要版本化、灰度、回滚和效果对比。

定位： Prompt 版本管理 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt 版本管理 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「Prompt 版本管理」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Prompt 版本管理 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「Prompt 版本管理」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt 版本管理 工程示例

1. 最小工程骨架

```
// Prompt版本管理: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt 版本管理」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Prompt 版本管理 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「Prompt 版本管理」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「Prompt 版本管理」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「Prompt 版本管理」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「Prompt 版本管理」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

- Prompt 版本管理 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt 版本管理 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt 版本管理」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt 评估集 核心讲解

本质：没有评估集就无法判断提示词是否变好。评估集要覆盖常见、边界和失败样本。

定位：Prompt 评估集 属于「Prompt 与结构化输出」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt 评估集 不是写一段“请你扮演专家”的话，而是在给模型设计一份接口协议。好的 Prompt 会明确角色、任务、上下文、约束、输出格式、失败策略和可验证证据。

1. 运行机制拆解

1. 角色只解决“用什么视角回答”，任务才解决“要完成什么动作”。如果角色和任务混在一起，Prompt 后续很难复用和评估。
2. 上下文是输入数据，不是越多越好。无关材料会扩大 token 成本，也会把模型注意力拉偏。
3. 结构化输出要先从业务字段反推：结论、原因、证据、风险、缺失信息、建议动作分别用什么字段表达。
4. Prompt Injection 的本质是用户把“数据”伪装成“指令”。防护不能只靠一句“不要听用户乱说”，而要靠输入隔离、工具权限和输出校验。

2. Java 工程落地抓手

- 用 PromptTemplate 管理模板字段，用版本号管理变更，用 JSON Schema 或 Java record 做输出契约。
- 把 promptId、version、变量值摘要和输出校验结果写入日志，方便回放问题。
- 为每个关键 Prompt 准备评估集，覆盖正常、边界、恶意输入和资料不足样本。

3. 核心架构图



关键误区：最危险的 Prompt 是看起来很长、很认真，但没有字段契约、没有评估集、没有版本号。它上线后不可治理。

判断是否学会：你能讲清「Prompt 评估集」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Prompt 评估集 组件拆解

1. 组件职责

1. 角色：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 任务：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 上下文：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 约束：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
5. 输出 Schema：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
6. 服务端校验：这是「Prompt 评估集」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt 评估集 工程示例

1. 最小工程骨架

```
// Prompt评估集: prompt contract
PromptTemplate template = promptRegistry.load(name, version);
String prompt = template.render(input, context, constraints);
JsonNode json = jsonRepair.parse(chatClient.call(prompt));
Result result = validator.validate(json);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt 评估集」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Prompt 评估集 生产坑与验收

1. 常见生产坑

1. 把提示词写成大作文：对「Prompt 评估集」来说，处理建议是：把角色、任务、上下文、约束、输出格式拆成模板字段，并用版本号管理。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只约束格式不约束业务：对「Prompt 评估集」来说，处理建议是：为字段写含义、类型、范围和缺失策略，服务端用 JSON Schema 或 Java Validator 校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有版本管理：对「Prompt 评估集」来说，处理建议是：Prompt 发布要走灰度、对照样本和回滚开关。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 忽略 Prompt Injection：对「Prompt 评估集」来说，处理建议是：把用户输入当数据而不是指令，用系统提示词、工具权限和输出校验共同防注入。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- JSON 合法率
- 字段完整率
- 业务规则命中率
- 拒答正确率
- 版本对比胜率

3. 学完必须能回答

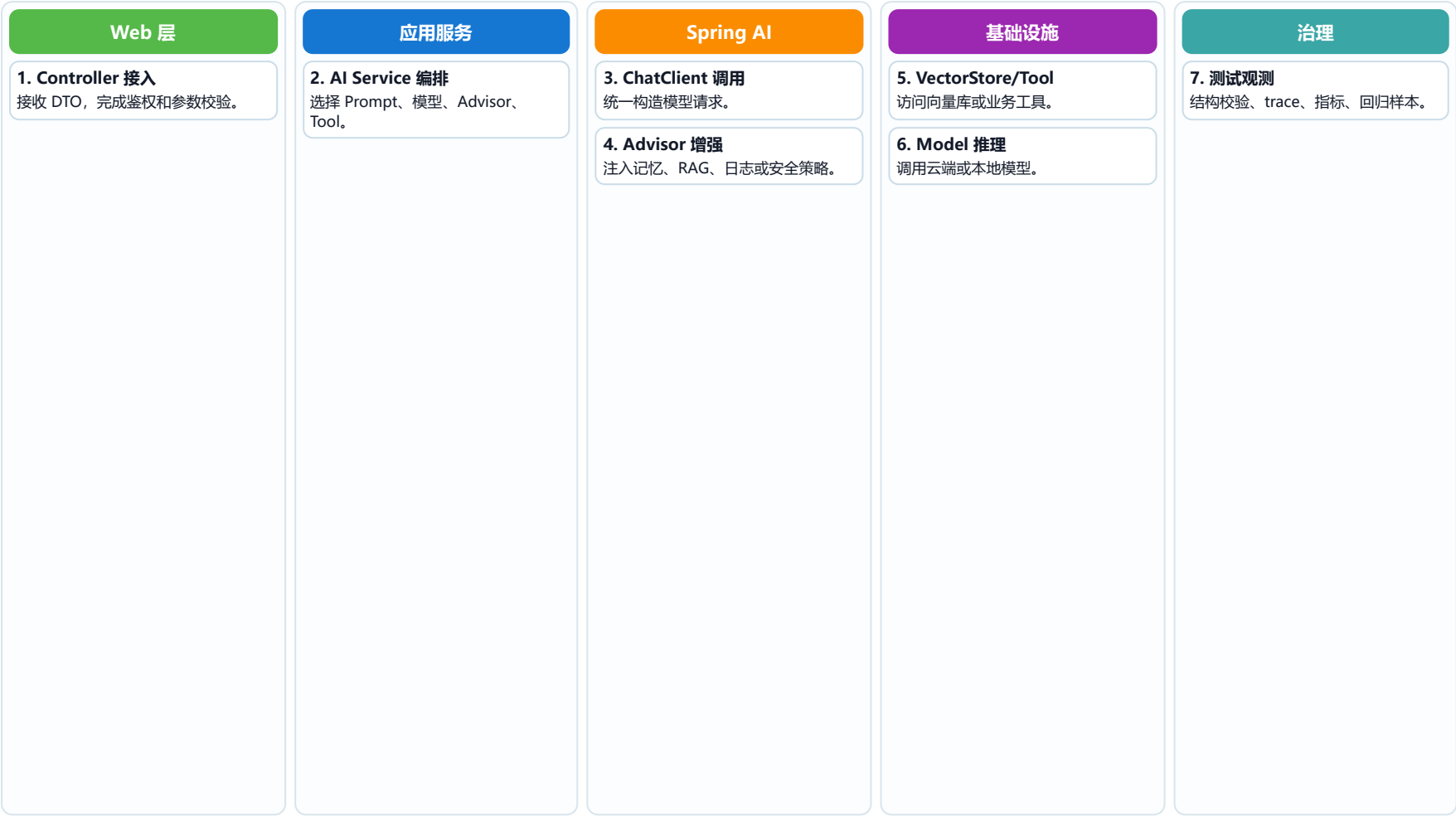
- Prompt 评估集的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt 评估集失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt 评估集」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Spring AI 入门 模块架构图

理解 Spring AI 的工程抽象，并放入 Spring Boot 分层架构。

这一章先看什么：Spring AI 工程抽象 不是背概念，而是看它在系统里怎么落位。先把 把框架当万能；Prompt 写在 Controller 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

ChatClient 抽象 核心讲解

本质： ChatClient 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位： ChatClient 抽象 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

ChatClient 抽象 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「ChatClient 抽象」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

ChatClient 抽象 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「ChatClient 抽象」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「ChatClient 抽象」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「ChatClient 抽象」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「ChatClient 抽象」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- ChatClient 抽象 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 ChatClient 抽象 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「ChatClient 抽象」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

ChatClient.Builder 核心讲解

本质： ChatClient.Builder 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位： ChatClient.Builder 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

ChatClient.Builder 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「ChatClient.Builder」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

ChatClient.Builder 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「ChatClient.Builder」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「ChatClient.Builder」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「ChatClient.Builder」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「ChatClient.Builder」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- ChatClient.Builder 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 ChatClient.Builder 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「ChatClient.Builder」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Model 抽象 核心讲解

本质：Model 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：Model 抽象 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Model 抽象 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「Model 抽象」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Model 抽象 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「Model 抽象」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「Model 抽象」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「Model 抽象」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「Model 抽象」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- Model 抽象 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Model 抽象 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Model 抽象」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Advisor 机制 核心讲解

本质：Advisor 机制 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：Advisor 机制 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Advisor 机制 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「Advisor 机制」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Advisor 机制 组件拆解

1. 组件职责

1. ChatClient: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. PromptTemplate: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. Advisor: 这是「Advisor 机制」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. ModelOptions: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
5. VectorStore: 负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Tool Callback: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Advisor 机制 工程示例

1. 最小工程骨架

```
// Advisor机制: Spring AI service
@Service
class AiApplicationService {
    Result handle(Request req) {
        return chatClient.prompt(promptFactory.create(req)).call().entity(Result.class);
    }
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Advisor 机制」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Advisor 机制 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「Advisor 机制」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「Advisor 机制」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「Advisor 机制」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「Advisor 机制」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- Advisor 机制 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Advisor 机制 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Advisor 机制」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

PromptTemplate 核心讲解

本质： PromptTemplate 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位： PromptTemplate 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

PromptTemplate 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「PromptTemplate」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

PromptTemplate 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「PromptTemplate」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「PromptTemplate」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「PromptTemplate」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「PromptTemplate」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- PromptTemplate 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 PromptTemplate 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「PromptTemplate」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

VectorStore 抽象 核心讲解

本质：VectorStore 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：VectorStore 抽象 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

VectorStore 抽象 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



VectorStore 抽象 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「VectorStore 抽象」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「VectorStore 抽象」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「VectorStore 抽象」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「VectorStore 抽象」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- VectorStore 抽象 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 VectorStore 抽象 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「VectorStore 抽象」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Tool Calling 集成 核心讲解

本质：Tool Calling 集成 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：Tool Calling 集成 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Tool Calling 集成 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「Tool Calling 集成」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Tool Calling 集成 组件拆解

1. 组件职责

1. ChatClient: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. PromptTemplate: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. Advisor: 这是「Tool Calling 集成」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. ModelOptions: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
5. VectorStore: 负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Tool Callback: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Tool Calling 集成 工程示例

1. 最小工程骨架

```
// ToolCalling集成: Spring AI service
@Service
class AiApplicationService {
    Result handle(Request req) {
        return chatClient.prompt(promptFactory.create(req)).call().entity(Result.class);
    }
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Tool Calling 集成」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Tool Calling 集成 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「Tool Calling 集成」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「Tool Calling 集成」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「Tool Calling 集成」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「Tool Calling 集成」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- Tool Calling 集成 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Tool Calling 集成 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Tool Calling 集成」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

自动配置与排除 核心讲解

本质：自动配置与排除 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：自动配置与排除 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

自动配置与排除 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



自动配置与排除 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「自动配置与排除」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「自动配置与排除」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「自动配置与排除」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「自动配置与排除」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- 自动配置与排除 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 自动配置与排除 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「自动配置与排除」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

AI 测试策略 核心讲解

本质：AI 测试策略 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：AI 测试策略 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

AI 测试策略 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



AI 测试策略 组件拆解

1. 组件职责

1. ChatClient: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. PromptTemplate: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. Advisor: 这是「AI 测试策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. ModelOptions: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
5. VectorStore: 负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Tool Callback: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

AI 测试策略 工程示例

1. 最小工程骨架

```
// AI测试策略: Spring AI service
@Service
class AiApplicationService {
    Result handle(Request req) {
        return chatClient.prompt(promptFactory.create(req)).call().entity(Result.class);
    }
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「AI 测试策略」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

AI 测试策略 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「AI 测试策略」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「AI 测试策略」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「AI 测试策略」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「AI 测试策略」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

- AI 测试策略 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 AI 测试策略 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「AI 测试策略」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

与现有项目融合 核心讲解

本质：与现有项目融合 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。

定位：与现有项目融合 属于「Spring AI 工程抽象」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

与现有项目融合 要放在 Spring Boot 分层架构里理解。Spring AI 的价值不是让你少写几行调用代码，而是把模型、Prompt、Advisor、VectorStore、Tool 变成可组合、可替换、可测试的工程组件。

1. 运行机制拆解

1. ChatClient 是面向应用层的调用入口，它把消息、参数、模型和响应解析组合起来，但不应该承担业务权限和事务。
2. Advisor 类似 AI 调用链的拦截器，可用于注入 RAG 上下文、记录日志、做安全检查或改写请求。链路越复杂，Advisor 的顺序越关键。
3. VectorStore、Tool Callback 和 ModelOptions 是基础设施能力，应该通过配置和 Bean 注入管理，而不是散落在业务方法里。
4. Spring AI 解决的是工程抽象，不解决业务正确性。RAG 准不准、工具安全吗、Prompt 稳不稳，仍然需要你设计。

2. Java 工程落地抓手

- 推荐 Controller -> Application Service -> AI Service -> Infrastructure Adapter 分层，AI Service 聚合 ChatClient、PromptFactory、Retriever、ToolRegistry。
- 测试不要只 mock 模型返回值，还要测结构化解析、Advisor 是否注入上下文、权限过滤是否生效、工具是否被拦截。
- 把模型配置、Prompt 版本、TopK、rerank 开关、工具白名单放入配置中心或数据库，支持灰度和回滚。

3. 核心架构图



关键误区：不要把 Spring AI 当成魔法框架。框架只给抽象和接线，真正的生产质量来自边界设计、数据治理和测试。

判断是否学会：你能讲清「与现有项目融合」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

与现有项目融合 组件拆解

1. 组件职责

1. ChatClient: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. PromptTemplate: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. Advisor: 这是「与现有项目融合」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. ModelOptions: 负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
5. VectorStore: 负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Tool Callback: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

与现有项目融合 工程示例

1. 最小工程骨架

```
// 与现有项目融合: Spring AI service
@Service
class AiApplicationService {
    Result handle(Request req) {
        return chatClient.prompt(promptFactory.create(req)).call().entity(Result.class);
    }
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「与现有项目融合」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

与现有项目融合 生产坑与验收

1. 常见生产坑

1. 把框架当万能：对「与现有项目融合」来说，处理建议是：把 AI 逻辑放到 Application Service，不要堆在 Controller。这类治理必须落到系统机制上，而不是停留在口头规范。
2. Prompt 写在 Controller：对「与现有项目融合」来说，处理建议是：Prompt、Advisor、Tool、VectorStore 都要可配置、可测试、可替换。这类治理必须落到系统机制上，而不是停留在口头规范。
3. Advisor 链路黑盒：对「与现有项目融合」来说，处理建议是：为 Advisor 增加调用链日志，记录注入了什么上下文、检索了什么内容。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有 AI 集成测试：对「与现有项目融合」来说，处理建议是：测试断言结构、引用、拒答、权限、工具调用和关键业务字段。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 调用成功率
- 结构化解析通过率
- Advisor 命中率
- 测试覆盖率
- 业务接口延迟

3. 学完必须能回答

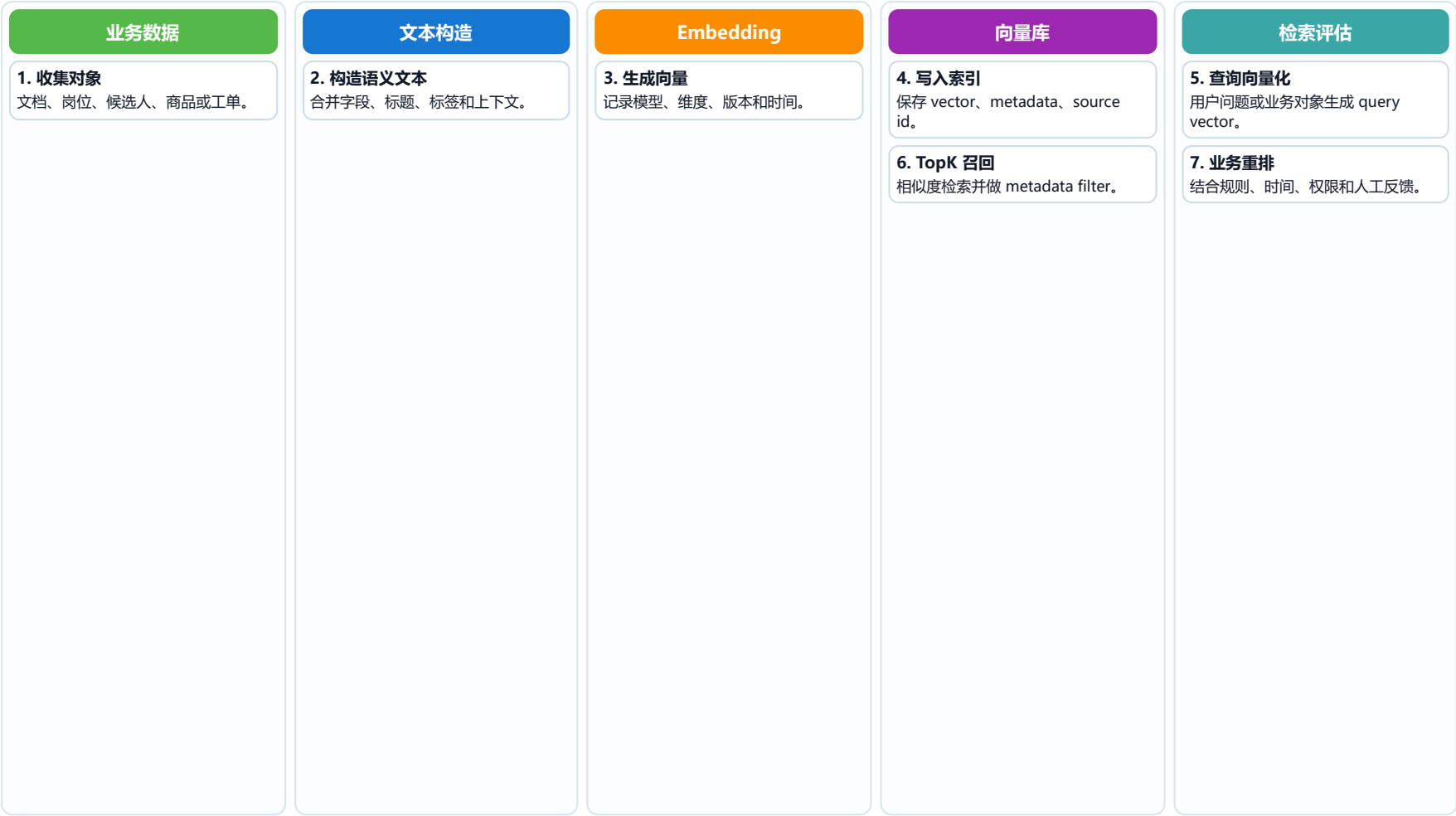
- 与现有项目融合 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 与现有项目融合 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「与现有项目融合」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Embedding 与向量库 模块架构图

理解语义检索底层逻辑：向量模型、文本构造、索引、元数据和评估。

这一章先看什么： Embedding 与向量库 不是背概念，而是看它在系统里怎么落位。先把 混用不同向量模型；只存正文不存元数据 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Embedding 本质 核心讲解

本质： Embedding 把文本映射到高维向量空间，让语义相近的文本在距离上更接近。

定位： Embedding 本质 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Embedding 本质 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



Embedding 本质 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「Embedding 本质」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Embedding 本质 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Embedding 本质 工程示例

1. 最小工程骨架

```
// Embedding本质: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Embedding 本质」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Embedding 本质 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「Embedding 本质」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「Embedding 本质」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「Embedding 本质」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「Embedding 本质」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- Embedding 本质 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Embedding 本质 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Embedding 本质」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

向量维度 核心讲解

本质：维度由 embedding 模型决定，不同模型向量空间不可混用。

定位：向量维度 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

向量维度 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「向量维度」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

向量维度 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「向量维度」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「向量维度」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「向量维度」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「向量维度」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- 向量维度 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 向量维度 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「向量维度」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

相似度度量 核心讲解

本质：常见度量有 cosine、dot product、L2，必须和模型训练方式及索引匹配。

定位：相似度度量 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

相似度度量的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「相似度量」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

相似度度量 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「相似度度量」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

相似度度量 工程示例

1. 最小工程骨架

```
// 相似度度量: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「相似度度量」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

相似度度量 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「相似度度量」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「相似度度量」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「相似度度量」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「相似度度量」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- 相似度度量 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 相似度度量 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「相似度度量」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

语义文本构造 核心讲解

本质：向量化前的文本组织决定召回质量，要把业务字段合成模型能理解的语义画像。

定位：语义文本构造 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

语义文本构造 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「语义文本构造」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

语义文本构造 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「语义文本构造」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

语义文本构造 工程示例

1. 最小工程骨架

```
// 语义文本构造: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「语义文本构造」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

语义文本构造 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「语义文本构造」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「语义文本构造」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「语义文本构造」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「语义文本构造」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- 语义文本构造 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 语义文本构造 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「语义文本构造」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Chunk 策略 核心讲解

本质： chunk 影响召回粒度。太大干扰多，太小语义不足，要按标题、段落和语义边界切。

定位： Chunk 策略 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Chunk 策略 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



Chunk 策略 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「Chunk 策略」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Chunk 策略 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Chunk 策略 工程示例

1. 最小工程骨架

```
// Chunk策略: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Chunk 策略」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Chunk 策略 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「Chunk 策略」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「Chunk 策略」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「Chunk 策略」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「Chunk 策略」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- Chunk 策略 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Chunk 策略 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Chunk 策略」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Metadata 设计 核心讲解

本质：元数据用于权限、过滤、溯源和版本治理，是生产级向量库关键。

定位：Metadata 设计 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Metadata 设计 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



Metadata 设计 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「Metadata 设计」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Metadata 设计 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Metadata 设计 工程示例

1. 最小工程骨架

```
// Metadata设计: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Metadata 设计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Metadata 设计 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「Metadata 设计」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「Metadata 设计」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「Metadata 设计」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「Metadata 设计」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- Metadata 设计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Metadata 设计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Metadata 设计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

pgvector 核心讲解

本质：pgvector 适合 Java 企业项目快速落地，能和 PostgreSQL 事务、SQL、权限结合。

定位：pgvector 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

pgvector 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「pgvector」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

pgvector 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「pgvector」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

pgvector 工程示例

1. 最小工程骨架

```
// pgvector: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「pgvector」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

pgvector 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「pgvector」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「pgvector」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「pgvector」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「pgvector」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- pgvector 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 pgvector 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「pgvector」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Milvus/Redis Vector 核心讲解

本质： Milvus 偏大规模检索，Redis Vector 偏低延迟和缓存场景。

定位： Milvus/Redis Vector 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Milvus/Redis Vector 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

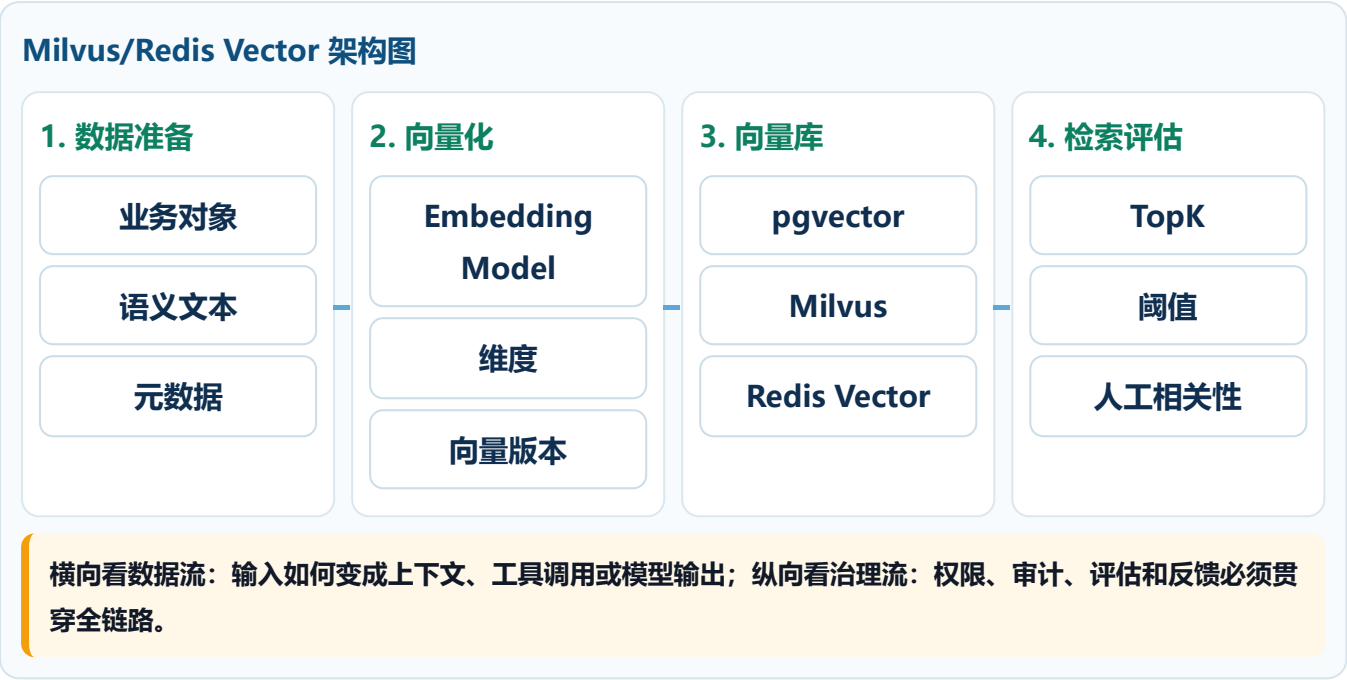
1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「Milvus/Redis Vector」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Milvus/Redis Vector 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「Milvus/Redis Vector」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「Milvus/Redis Vector」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「Milvus/Redis Vector」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「Milvus/Redis Vector」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- Milvus/Redis Vector 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Milvus/Redis Vector 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Milvus/Redis Vector」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

TopK 与阈值 核心讲解

本质：TopK 不是越大越好，阈值要通过评估集校准。

定位：TopK 与阈值 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

TopK 与阈值 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「TopK 与阈值」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

TopK 与阈值 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「TopK 与阈值」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

TopK 与阈值 工程示例

1. 最小工程骨架

```
// TopK与阈值: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「TopK 与阈值」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

TopK 与阈值 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「TopK 与阈值」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「TopK 与阈值」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「TopK 与阈值」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「TopK 与阈值」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

- TopK 与阈值 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 TopK 与阈值 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「TopK 与阈值」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

语义搜索评估 核心讲解

本质：评估要看是否搜到正确答案、是否排序靠前、是否带正确元数据。

定位：语义搜索评估 属于「Embedding 与向量库」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

语义搜索评估 的核心是把文本放进一个语义空间，让“意思相近”可以用向量距离近似计算。它不是数据库索引的替代品，而是召回候选事实的一种信号。

1. 运行机制拆解

1. Embedding 模型会把文本压缩成固定维度向量。维度、训练语料和归一化方式不同，向量空间就不同，不能混用。
2. 语义文本构造比很多人想象得重要：标题、业务字段、同义词、时间、状态、所属组织是否进入文本，会直接影响召回。
3. 向量库检索先给候选，不给最终答案。TopK、阈值、metadata filter 和 rerank 共同决定哪些片段能进入模型上下文。
4. 元数据是生产生命线：来源、租户、权限、版本、时间、对象 ID、文档类型缺失时，检索结果即使相似也可能不能用。

2. Java 工程落地抓手

- 向量表至少保存 id、vector、content、metadata、embeddingModel、embeddingVersion、sourceId、updatedAt。
- 文档重建向量时要按模型版本隔离集合，避免新旧向量混在同一索引中。
- 用 Recall@K、MRR、人工相关性评分评估语义搜索，不要只凭几次查询感觉“还行”。

3. 核心架构图



关键误区：不要把相似度分数当业务正确性。相似只说明“语义像”，不说明“有权限、最新、权威、能回答”。

判断是否学会：你能讲清「语义搜索评估」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

语义搜索评估 组件拆解

1. 组件职责

1. Embedding Model：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. 语义文本：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. 向量空间：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. 相似度度量：这是「语义搜索评估」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
5. 索引：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. 元数据过滤：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

语义搜索评估 工程示例

1. 最小工程骨架

```
// 语义搜索评估: semantic retrieval
String text = semanticTextBuilder.build(object);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(id, vector, metadata);
List<Match> matches = vectorStore.search(queryVector, filter, topK);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「语义搜索评估」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

语义搜索评估 生产坑与验收

1. 常见生产坑

1. 混用不同向量模型：对「语义搜索评估」来说，处理建议是：向量表记录 embedding model、维度、版本和生成时间，不同模型向量分集合存储。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只存正文不存元数据：对「语义搜索评估」来说，处理建议是：元数据至少包含来源、租户、权限、版本、时间、业务类型和可追溯 ID。这类治理必须落到系统机制上，而不是停留在口头规范。
3. chunk 粒度不合适：对「语义搜索评估」来说，处理建议是：根据文档结构切分，表格、标题、条款、代码块使用不同策略。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 把相似度当业务评分：对「语义搜索评估」来说，处理建议是：相似度只作为召回信号，最终排序结合业务规则、时间、权限、质量分和反馈。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- Recall@K
- MRR
- 命中率
- 权限过滤正确率
- 人工相关性评分

3. 学完必须能回答

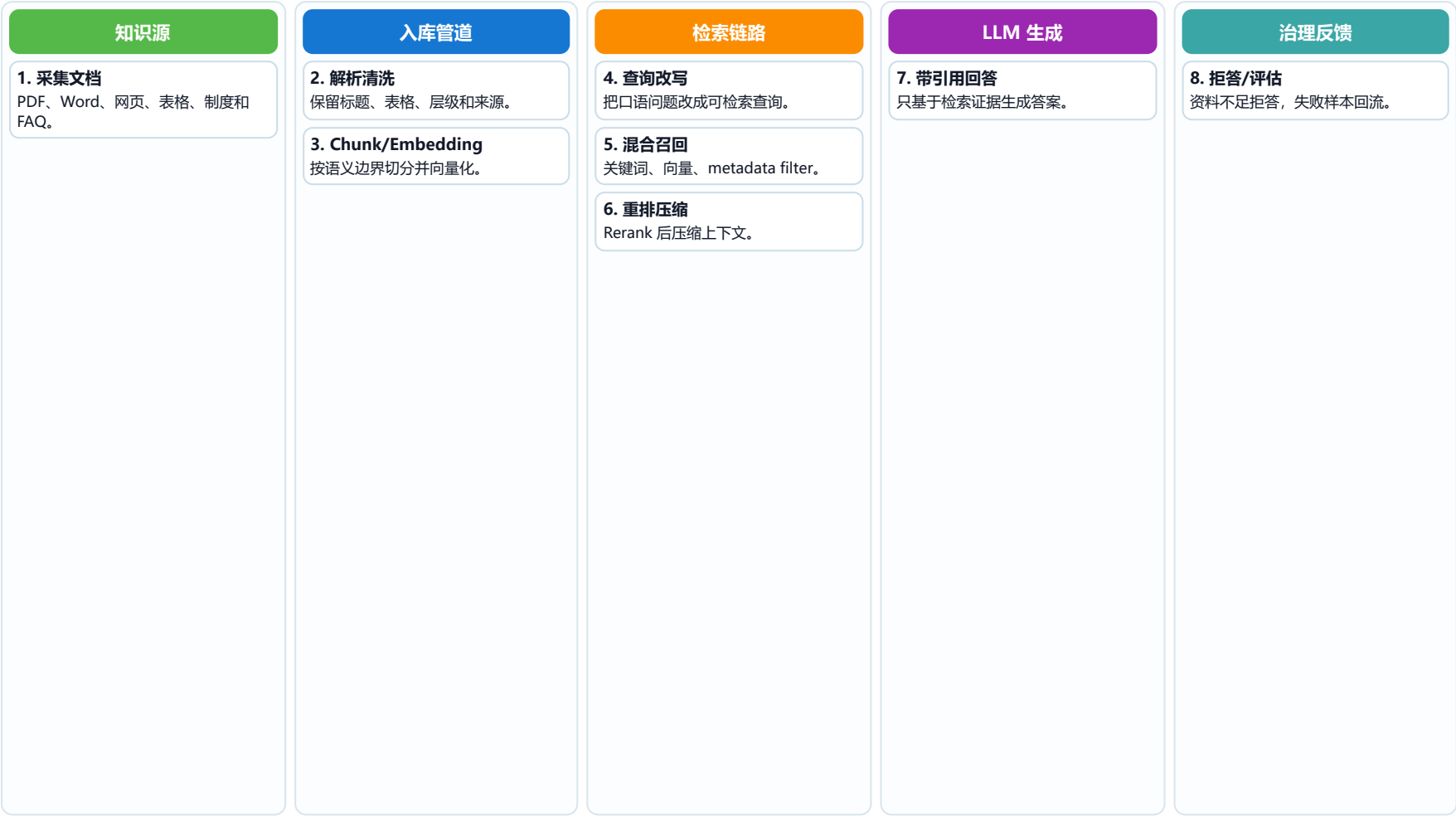
- 语义搜索评估 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 语义搜索评估 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「语义搜索评估」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

RAG 知识库 v1 模块架构图

搭建最小可用 RAG 链路：采集、解析、切分、检索、引用和拒答。

这一章先看什么：RAG 知识检索增强 不是背概念，而是看它在系统里怎么落位。先把 整篇文档塞给模型；没有引用来源 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

RAG 第一性原理 核心讲解

本质：RAG 第一性原理 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：RAG 第一性原理 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

RAG 第一性原理 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区： RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会： 你能讲清「RAG 第一性原理」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

RAG 第一性原理 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

RAG 第一性原理 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

RAG 第一性原理 工程示例

1. 最小工程骨架

```
// RAG第一性原理: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「RAG 第一性原理」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

RAG 第一性原理 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「RAG 第一性原理」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「RAG 第一性原理」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「RAG 第一性原理」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「RAG 第一性原理」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- RAG 第一性原理 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 RAG 第一性原理 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「RAG 第一性原理」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

文档采集 核心讲解

本质：文档采集 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：文档采集 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

文档采集 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



文档采集 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

文档采集 工程示例

1. 最小工程骨架

```
// 文档采集: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「文档采集」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

文档采集 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「文档采集」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「文档采集」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「文档采集」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「文档采集」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 文档采集 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 文档采集 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「文档采集」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

文档解析 核心讲解

本质：文档解析 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：文档解析 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

文档解析 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



文档解析 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

文档解析 工程示例

1. 最小工程骨架

```
// 文档解析: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「文档解析」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

文档解析 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「文档解析」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「文档解析」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「文档解析」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「文档解析」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 文档解析 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 文档解析 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「文档解析」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

内容清洗 核心讲解

本质：内容清洗 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：内容清洗 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

内容清洗 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区： RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会： 你能讲清「内容清洗」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

内容清洗 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

内容清洗 工程示例

1. 最小工程骨架

```
// 内容清洗: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「内容清洗」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

内容清洗 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「内容清洗」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「内容清洗」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「内容清洗」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「内容清洗」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 内容清洗 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 内容清洗 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「内容清洗」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

切分与重叠 核心讲解

本质：切分与重叠 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：切分与重叠 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

切分与重叠 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「切分与重叠」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

切分与重叠 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

切分与重叠 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

切分与重叠 工程示例

1. 最小工程骨架

```
// 切分与重叠: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「切分与重叠」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

切分与重叠 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「切分与重叠」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「切分与重叠」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「切分与重叠」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「切分与重叠」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 切分与重叠 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 切分与重叠 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「切分与重叠」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

检索器 Retriever 核心讲解

本质：检索器 Retriever 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：检索器 Retriever 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

检索器 Retriever 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「检索器 Retriever」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

检索器 Retriever 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

检索器 Retriever 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

检索器 Retriever 工程示例

1. 最小工程骨架

```
// 检索器Retriever: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「检索器 Retriever」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

检索器 Retriever 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「检索器 Retriever」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「检索器 Retriever」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「检索器 Retriever」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「检索器 Retriever」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 检索器 Retriever 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 检索器 Retriever 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「检索器 Retriever」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

上下文组装 核心讲解

本质：上下文组装 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：上下文组装 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

上下文组装 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



上下文组装 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

上下文组装 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

上下文组装 工程示例

1. 最小工程骨架

```
// 上下文组装: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「上下文组装」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

上下文组装 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「上下文组装」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「上下文组装」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「上下文组装」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「上下文组装」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 上下文组装 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 上下文组装 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「上下文组装」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

引用来源 核心讲解

本质：引用来源 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：引用来源 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

引用来源 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



引用来源 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

引用来源 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

引用来源 工程示例

1. 最小工程骨架

```
// 引用来源: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「引用来源」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

引用来源 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「引用来源」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「引用来源」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「引用来源」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「引用来源」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 引用来源 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 引用来源 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「引用来源」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

资料不足拒答 核心讲解

本质：资料不足拒答 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：资料不足拒答 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

资料不足拒答 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



资料不足拒答 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

资料不足拒答 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

资料不足拒答 工程示例

1. 最小工程骨架

```
// 资料不足拒答: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「资料不足拒答」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

资料不足拒答 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「资料不足拒答」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「资料不足拒答」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「资料不足拒答」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「资料不足拒答」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 资料不足拒答 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 资料不足拒答 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「资料不足拒答」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

增量更新 核心讲解

本质：增量更新 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。

定位：增量更新 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

增量更新 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



增量更新 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

增量更新 工程示例

1. 最小工程骨架

```
// 增量更新: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「增量更新」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

增量更新 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「增量更新」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「增量更新」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「增量更新」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「增量更新」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

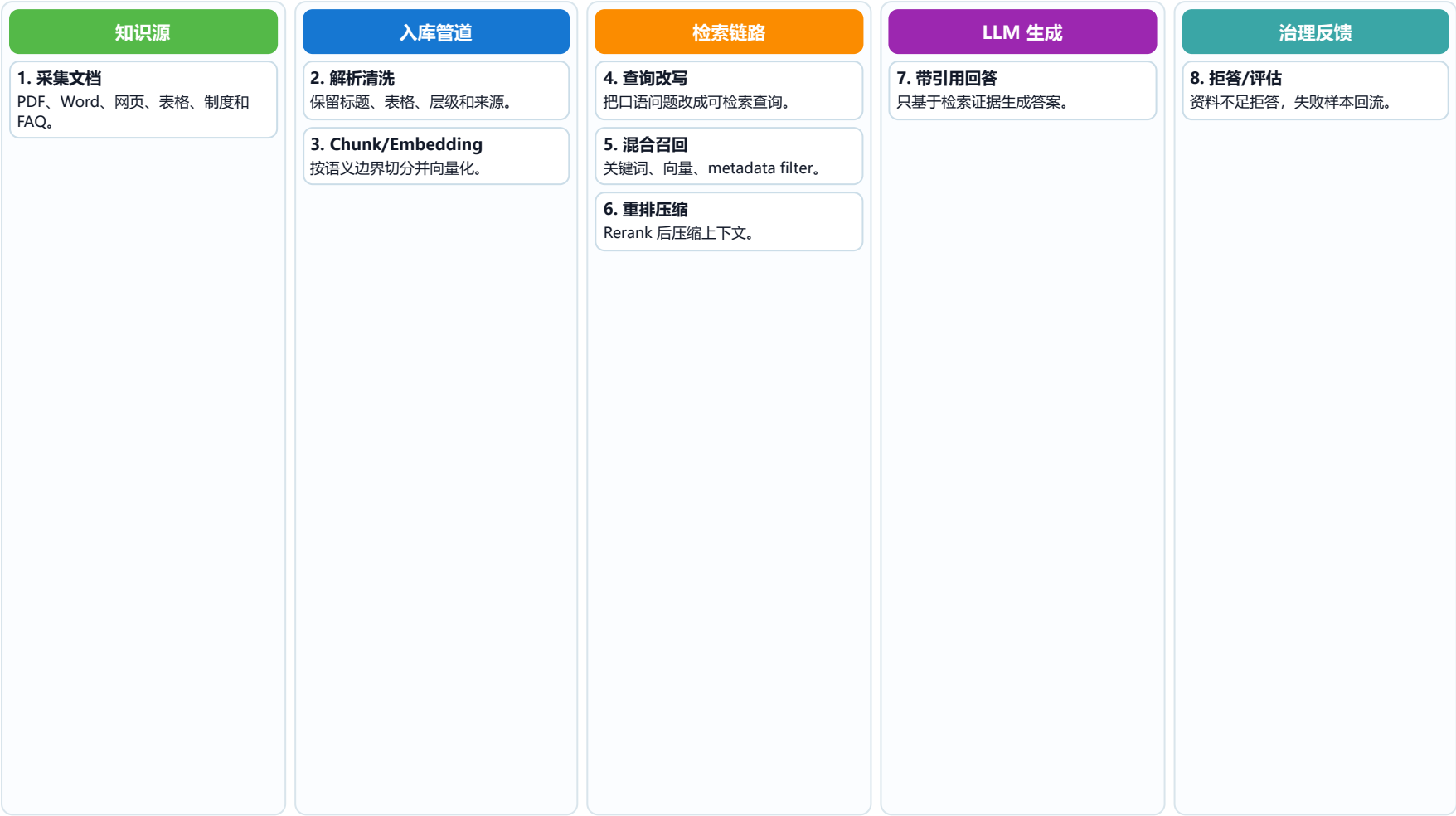
- 增量更新 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 增量更新 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「增量更新」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

RAG 工程优化 模块架构图

把能用的 RAG 变成更准、更稳、更可解释的 RAG。

这一章先看什么：RAG 知识检索增强 不是背概念，而是看它在系统里怎么落位。先把 整篇文档塞给模型；没有引用来源 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Query Rewrite 核心讲解

本质：Query Rewrite 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Query Rewrite 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Query Rewrite 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「Query Rewrite」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Query Rewrite 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Query Rewrite 工程示例

1. 最小工程骨架

```
// QueryRewrite: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Query Rewrite」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Query Rewrite 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Query Rewrite」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Query Rewrite」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Query Rewrite」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Query Rewrite」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Query Rewrite 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Query Rewrite 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Query Rewrite」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

混合检索 核心讲解

本质：混合检索 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：混合检索 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

混合检索 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「混合检索」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

混合检索 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

混合检索 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

混合检索 工程示例

1. 最小工程骨架

```
// 混合检索: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「混合检索」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

混合检索 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「混合检索」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「混合检索」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「混合检索」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「混合检索」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 混合检索 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 混合检索 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「混合检索」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Rerank 核心讲解

本质：Rerank 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Rerank 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Rerank 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「Rerank」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Rerank 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Rerank 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Rerank 工程示例

1. 最小工程骨架

```
// Rerank: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Rerank」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Rerank 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Rerank」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Rerank」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Rerank」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Rerank」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Rerank 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Rerank 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Rerank」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Metadata Filter 核心讲解

本质：Metadata Filter 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Metadata Filter 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Metadata Filter 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区： RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会： 你能讲清「Metadata Filter」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Metadata Filter 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Metadata Filter 工程示例

1. 最小工程骨架

```
// MetadataFilter: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Metadata Filter」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Metadata Filter 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Metadata Filter」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Metadata Filter」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Metadata Filter」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Metadata Filter」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Metadata Filter 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Metadata Filter 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Metadata Filter」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Parent-child Chunk 核心讲解

本质：Parent-child Chunk 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Parent-child Chunk 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Parent-child Chunk 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「Parent-child Chunk」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Parent-child Chunk 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Parent-child Chunk 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Parent-child Chunk 工程示例

1. 最小工程骨架

```
// Parent-childChunk: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Parent-child Chunk」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Parent-child Chunk 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Parent-child Chunk」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Parent-child Chunk」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Parent-child Chunk」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Parent-child Chunk」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Parent-child Chunk 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Parent-child Chunk 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Parent-child Chunk」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Context Compression 核心讲解

本质：Context Compression 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Context Compression 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Context Compression 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



Context Compression 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Context Compression 工程示例

1. 最小工程骨架

```
// ContextCompression: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Context Compression」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Context Compression 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Context Compression」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Context Compression」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Context Compression」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Context Compression」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Context Compression 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Context Compression 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Context Compression」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Table RAG 核心讲解

本质：Table RAG 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Table RAG 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Table RAG 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「Table RAG」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Table RAG 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Table RAG 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Table RAG 工程示例

1. 最小工程骨架

```
// TableRAG: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Table RAG」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Table RAG 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Table RAG」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Table RAG」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Table RAG」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Table RAG」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Table RAG 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Table RAG 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Table RAG」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Graph RAG 核心讲解

本质：Graph RAG 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：Graph RAG 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Graph RAG 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区：RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会：你能讲清「Graph RAG」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Graph RAG 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Graph RAG 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Graph RAG 工程示例

1. 最小工程骨架

```
// GraphRAG: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Graph RAG」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Graph RAG 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「Graph RAG」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「Graph RAG」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「Graph RAG」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「Graph RAG」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- Graph RAG 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Graph RAG 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Graph RAG」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

RAG 评估集 核心讲解

本质：RAG 评估集 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：RAG 评估集 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

RAG 评估集 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



关键误区： RAG 最大误区是只调 TopK。真正要调的是文档结构、chunk、元数据、检索策略、重排、上下文压缩和答案证据约束。

判断是否学会： 你能讲清「RAG 评估集」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

RAG 评估集 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

RAG 评估集 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

RAG 评估集 工程示例

1. 最小工程骨架

```
// RAG评估集: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「RAG 评估集」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

RAG 评估集 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「RAG 评估集」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「RAG 评估集」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「RAG 评估集」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「RAG 评估集」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- RAG 评估集 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 RAG 评估集 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「RAG 评估集」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

线上反馈闭环 核心讲解

本质：线上反馈闭环 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。

定位：线上反馈闭环 属于「RAG 知识检索增强」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

线上反馈闭环 的本质是让模型基于外部证据回答，而不是依赖训练记忆瞎猜。RAG 不是“向量库 + 大模型”，而是一条从资料治理到答案引用的证据流水线。

1. 运行机制拆解

1. 离线链路决定知识质量：采集、解析、清洗、切分、去重、入库、版本管理，每一步都会影响线上回答。
2. 在线链路决定回答质量：query rewrite、召回、metadata filter、rerank、context compression、生成和引用校验要配合。
3. chunk 策略是 RAG 的地基。太大导致噪声多，太小导致语义断裂；标题层级、表格、列表、代码和制度条款应采用不同策略。
4. 资料不足时拒答是能力，不是失败。生产 RAG 必须能说明缺少什么资料，而不是编一个看起来合理的答案。

2. Java 工程落地抓手

- 建立 DocumentIngestionJob，把解析结果、chunk、embedding、metadata、source version 全部可追踪。
- 回答对象包含 answer、citations、confidence、missingInfo、usedChunks，前端能点回原文。
- 评估集按问题、标准答案、期望引用、拒答条件组织，持续回归测试检索和生成策略。

3. 核心架构图



线上反馈闭环 组件拆解

1. 组件职责

1. Ingestion：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
2. Parser：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
3. Chunk：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
4. Retriever：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. Rerank：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
6. Generator：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

线上反馈闭环 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

线上反馈闭环 工程示例

1. 最小工程骨架

```
// 线上反馈闭环: RAG pipeline
String query = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(query, metadataFilter);
List<Chunk> top = reranker.rank(query, candidates).take(5);
Answer answer = generator.answer(question, contextWithCitations(top));
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「线上反馈闭环」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

线上反馈闭环 生产坑与验收

1. 常见生产坑

1. 整篇文档塞给模型：对「线上反馈闭环」来说，处理建议是：建立文档解析和 chunk 流水线，控制进入模型的片段数量、长度和来源。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有引用来源：对「线上反馈闭环」来说，处理建议是：答案必须带来源编号、标题、版本和片段 ID，前端允许点回原文验证。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 资料不足还强答：对「线上反馈闭环」来说，处理建议是：检索分数低、冲突来源多或无命中时拒答，并说明缺少什么资料。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有权限过滤：对「线上反馈闭环」来说，处理建议是：检索前带租户、部门、地区、角色、有效期等 metadata filter。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 召回正确率
- 引用正确率
- 拒答正确率
- 答案 groundedness
- 用户采纳率

3. 学完必须能回答

- 线上反馈闭环 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 线上反馈闭环 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「线上反馈闭环」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Tool Calling 模块架构图

让模型调用 Java 业务能力，但执行权永远在应用系统手里。

这一章先看什么：Tool Calling 与 MCP 不是背概念，而是看它在系统里怎么落位。先把 暴露大而全 Service；让模型直接传 SQL 这类问题看懂，再回头看图，才知道每条线为什么这么连。

MCP 协议 - 一张图连通

工具方实现一次，多个 AI 应用复用；应用侧统一管理权限、上下文和工具调用。

Host 宿主：Claude Code / Cursor / IDE / 企业 AI 平台

管理多个 Client，负责权限把关、上下文汇总、能力暴露给模型

Client A

Client B

Client C

文件 Server

本地文件系统

GitHub Server

Issue / PR / 代码

Postgres Server

查库只读

Tools 工具

tools/call

Resources 资源

只读 URI

Prompts 提示

用户控制 / 模板

本地：studio 子进程

Server 跑在哪？

远程：HTTP 端点

MCP 解决的不是“怎么调工具”，而是“工具如何被所有 AI 应用安全复用”。核心关键词：N x M、三角色、JSON-RPC、双向 Sampling、安全边界。

主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Tool Calling 本质 核心讲解

本质：Tool Calling 本质 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：Tool Calling 本质 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Tool Calling 本质 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「Tool Calling 本质」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Tool Calling 本质 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「Tool Calling 本质」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Tool Calling 本质 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Tool Calling 本质 工程示例

1. 最小工程骨架

```
// ToolCalling本质: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Tool Calling 本质」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Tool Calling 本质 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「Tool Calling 本质」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「Tool Calling 本质」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「Tool Calling 本质」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「Tool Calling 本质」来说，处理建议是：记录调用人、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- Tool Calling 本质 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Tool Calling 本质 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Tool Calling 本质」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

工具 Schema 核心讲解

本质：工具 Schema 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：工具 Schema 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

工具 Schema 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「工具 Schema」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

工具 Schema 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「工具 Schema」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

工具 Schema 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

工具 Schema 工程示例

1. 最小工程骨架

```
// 工具Schema: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「工具 Schema」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

工具 Schema 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「工具 Schema」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「工具 Schema」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「工具 Schema」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「工具 Schema」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 工具 Schema 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 工具 Schema 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「工具 Schema」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

参数校验 核心讲解

本质：参数校验 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：参数校验 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

参数校验 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「参数校验」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

参数校验 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「参数校验」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

参数校验 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

参数校验 工程示例

1. 最小工程骨架

```
// 参数校验: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「参数校验」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

参数校验 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「参数校验」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「参数校验」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「参数校验」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「参数校验」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 参数校验 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 参数校验 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「参数校验」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

只读优先 核心讲解

本质：只读优先 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：只读优先 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

只读优先 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「只读优先」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

只读优先 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「只读优先」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

只读优先 工程示例

1. 最小工程骨架

```
// 只读优先: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「只读优先」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

只读优先 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「只读优先」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「只读优先」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「只读优先」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「只读优先」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 只读优先 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 只读优先 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「只读优先」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

工具结果摘要 核心讲解

本质：工具结果摘要 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：工具结果摘要 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

工具结果摘要 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「工具结果摘要」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

工具结果摘要 组件拆解

1. 组件职责

- 1. Tool Schema：负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 2. 参数生成：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 3. 权限校验：负责让「工具结果摘要」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 业务执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 结果摘要：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

工具结果摘要 工程示例

1. 最小工程骨架

```
// 工具结果摘要: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「工具结果摘要」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

工具结果摘要 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「工具结果摘要」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「工具结果摘要」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「工具结果摘要」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「工具结果摘要」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 工具结果摘要 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 工具结果摘要 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「工具结果摘要」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

工具审计 核心讲解

本质：工具审计 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：工具审计 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

工具审计 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「工具审计」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

工具审计 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「工具审计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

工具审计 工程示例

1. 最小工程骨架

```
// 工具审计: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「工具审计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

工具审计 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「工具审计」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「工具审计」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「工具审计」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「工具审计」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 工具审计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 工具审计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「工具审计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

MCP Server 核心讲解

本质： MCP Server 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位： MCP Server 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

MCP Server 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「MCP Server」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

MCP Server 组件拆解

1. 组件职责

- 1. Tool Schema：负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 2. 参数生成：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 3. 权限校验：负责让「MCP Server」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 业务执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 结果摘要：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

MCP Server 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

MCP Server 工程示例

1. 最小工程骨架

```
// MCPServer: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「MCP Server」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

MCP Server 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「MCP Server」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「MCP Server」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「MCP Server」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「MCP Server」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- MCP Server 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 MCP Server 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「MCP Server」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Tool Registry 核心讲解

本质：Tool Registry 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：Tool Registry 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Tool Registry 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「Tool Registry」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Tool Registry 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「Tool Registry」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Tool Registry 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Tool Registry 工程示例

1. 最小工程骨架

```
// ToolRegistry: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Tool Registry」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Tool Registry 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「Tool Registry」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「Tool Registry」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「Tool Registry」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「Tool Registry」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- Tool Registry 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Tool Registry 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Tool Registry」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

工具失败恢复 核心讲解

本质：工具失败恢复 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：工具失败恢复 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

工具失败恢复 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「工具失败恢复」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

工具失败恢复 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型 “看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「工具失败恢复」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

工具失败恢复 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

工具失败恢复 工程示例

1. 最小工程骨架

```
// 工具失败恢复: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「工具失败恢复」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

工具失败恢复 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「工具失败恢复」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「工具失败恢复」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「工具失败恢复」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「工具失败恢复」来说，处理建议是：记录调用入、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

- 工具失败恢复 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 工具失败恢复 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「工具失败恢复」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Human Confirmation 核心讲解

本质：Human Confirmation 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。

定位：Human Confirmation 属于「Tool Calling 与 MCP」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Human Confirmation 的核心不是让模型“拥有工具”，而是让模型在受控边界内提出工具调用意图，由 Java 系统负责校验和执行。MCP 进一步把工具连接标准化，减少每个应用重复对接。

1. 运行机制拆解

1. 模型看到的是工具名称、描述和参数 schema，它生成的是“建议调用哪个工具、带哪些参数”，不是直接执行权。
2. 服务端必须校验参数类型、枚举范围、用户权限、租户范围、风险等级和幂等性。模型生成的任何参数都不能直接信任。
3. 工具结果要变成 observation 再回给模型。结果过大时先摘要、裁剪和脱敏，只返回回答所需字段。
4. MCP 的角色是 Host、Client、Server。Host 管理上下文和权限，Client 与 Server 通信，Server 暴露 tools/resources/prompts。

2. Java 工程落地抓手

- 工具优先设计成只读、小粒度、单职责，例如 queryOrderStatus，不要暴露 doEverythingService。
- 高风险动作必须 Human Confirmation，例如发通知、改状态、扣费、审批、删除、外部提交。
- 每次工具调用记录 traceId、toolName、argsDigest、policyDecision、duration、resultSummary 和风险等级。

3. 核心架构图



关键误区：最危险的做法是把内部 Service 全量暴露给模型。模型负责建议，系统负责执行边界，这条线不能模糊。

判断是否学会：你能讲清「Human Confirmation」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Human Confirmation 组件拆解

1. 组件职责

1. Tool Schema: 负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
2. 参数生成: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
3. 权限校验: 负责让「Human Confirmation」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 业务执行: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 结果摘要: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 审计日志: 负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Human Confirmation 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Human Confirmation 工程示例

1. 最小工程骨架

```
// HumanConfirmation: safe tool calling
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = validator.validate(call.args(), user, tenant);
ToolResult result = toolExecutor.executeReadOnly(args);
audit.save(call, args, result);
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Human Confirmation」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Human Confirmation 生产坑与验收

1. 常见生产坑

1. 暴露大而全 Service：对「Human Confirmation」来说，处理建议是：工具命名和描述要单一明确，大工具拆成多个只读小工具。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 让模型直接传 SQL：对「Human Confirmation」来说，处理建议是：禁止模型直接传 SQL、脚本或自由表名，参数用枚举、ID、范围和权限白名单校验。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 工具结果过大：对「Human Confirmation」来说，处理建议是：工具结果先摘要再给模型，只返回回答所需字段。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有工具审计：对「Human Confirmation」来说，处理建议是：记录调用人、参数、权限判定、结果摘要、耗时和风险等级，高风险动作人工确认。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 工具选择准确率
- 参数校验通过率
- 工具失败率
- 人工确认率
- 审计完整率

3. 学完必须能回答

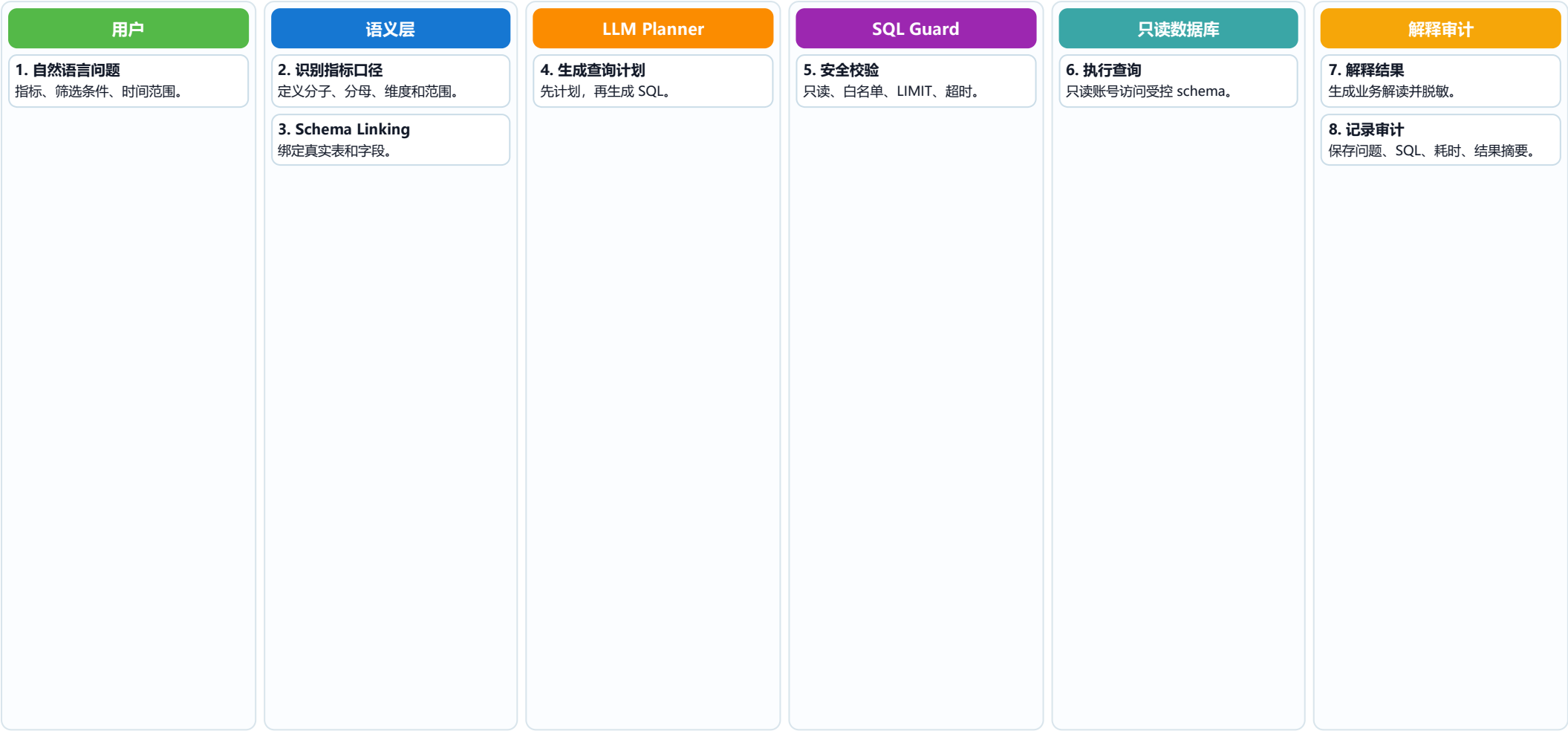
- Human Confirmation 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Human Confirmation 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Human Confirmation」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

数据库问答助手 模块架构图

NL2SQL 价值高也风险高，必须用语义层、安全层和审计层包住。

这一章先看什么：数据库问答 不是背概念，而是看它在系统里怎么落位。先把 高权限账号执行 SQL；没有表字段白名单 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

NL2SQL 本质 核心讲解

本质： NL2SQL 本质 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位： NL2SQL 本质 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

NL2SQL 本质 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



NL2SQL 本质 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

NL2SQL 本质 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

NL2SQL 本质 工程示例

1. 最小工程骨架

```
// NL2SQL本质: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「NL2SQL 本质」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

NL2SQL 本质 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「NL2SQL 本质」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「NL2SQL 本质」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「NL2SQL 本质」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「NL2SQL 本质」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- NL2SQL 本质 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 NL2SQL 本质 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「NL2SQL 本质」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

语义层 核心讲解

本质：语义层 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：语义层 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

语义层 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



语义层 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

语义层 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

语义层 工程示例

1. 最小工程骨架

```
// 语义层: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「语义层」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

语义层 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「语义层」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「语义层」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「语义层」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「语义层」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- 语义层 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 语义层 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「语义层」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Schema Linking 核心讲解

本质：Schema Linking 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：Schema Linking 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Schema Linking 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



Schema Linking 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Schema Linking 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Schema Linking 工程示例

1. 最小工程骨架

```
// SchemaLinking: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Schema Linking」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Schema Linking 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「Schema Linking」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「Schema Linking」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「Schema Linking」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「Schema Linking」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- Schema Linking 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Schema Linking 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Schema Linking」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

SQL 生成计划 核心讲解

本质：SQL 生成计划 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：SQL 生成计划 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

SQL 生成计划 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



SQL 生成计划 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

SQL 生成计划 工程示例

1. 最小工程骨架

```
// SQL生成计划: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「SQL 生成计划」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

SQL 生成计划 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「SQL 生成计划」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「SQL 生成计划」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「SQL 生成计划」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「SQL 生成计划」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- SQL 生成计划 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 SQL 生成计划 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「SQL 生成计划」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

SQL Guard 核心讲解

本质：SQL Guard 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：SQL Guard 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

SQL Guard 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



SQL Guard 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

SQL Guard 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

SQL Guard 工程示例

1. 最小工程骨架

```
// SQLGuard: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「SQL Guard」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

SQL Guard 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「SQL Guard」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「SQL Guard」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「SQL Guard」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「SQL Guard」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- SQL Guard 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 SQL Guard 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「SQL Guard」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

只读账号 核心讲解

本质：只读账号 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：只读账号 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

只读账号 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

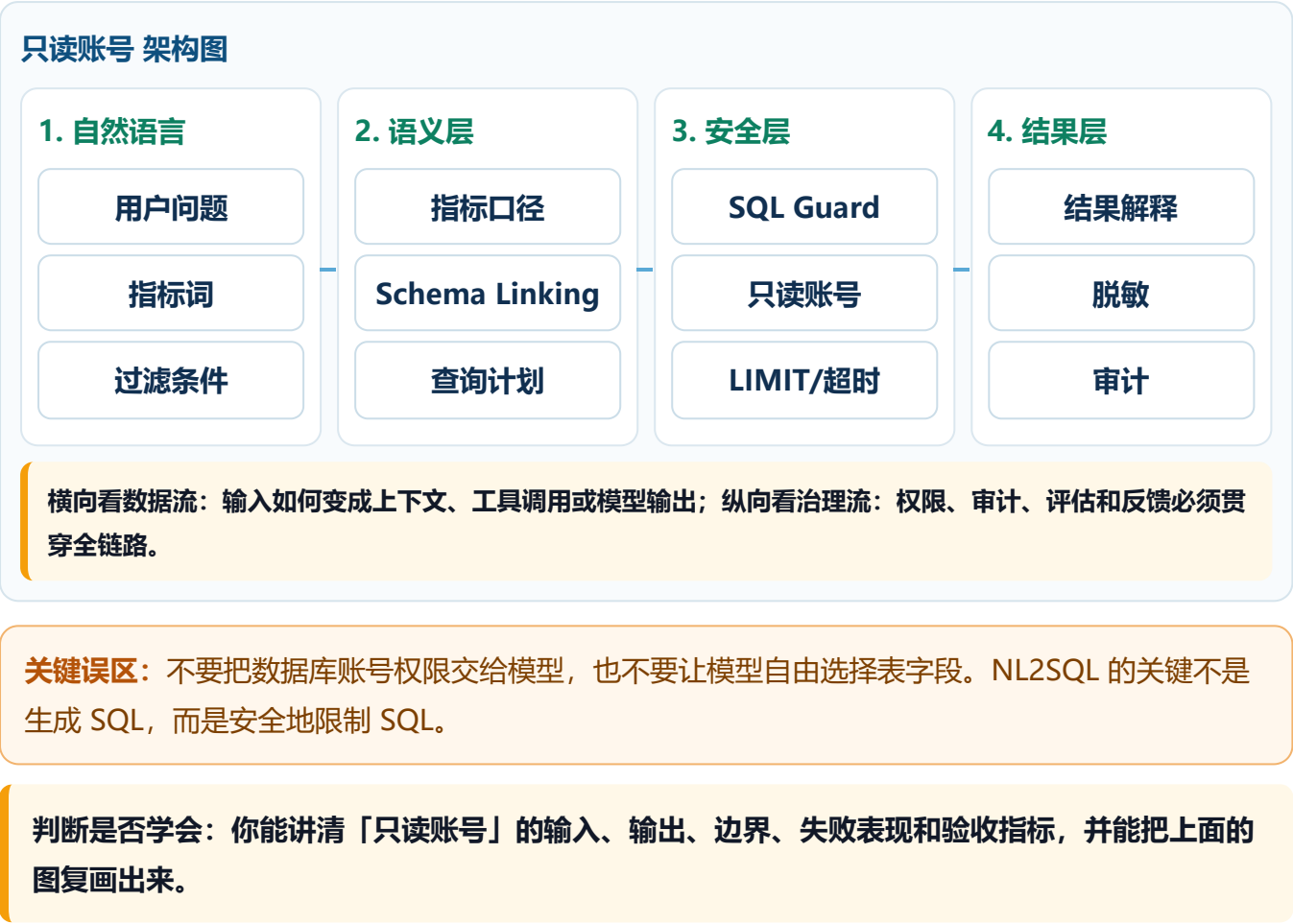
1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



只读账号 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

只读账号 工程示例

1. 最小工程骨架

```
// 只读账号: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「只读账号」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

只读账号 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「只读账号」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「只读账号」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「只读账号」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「只读账号」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- 只读账号 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 只读账号 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「只读账号」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

分页和超时 核心讲解

本质：分页和超时 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：分页和超时 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

分页和超时 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



分页和超时 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

分页和超时 工程示例

1. 最小工程骨架

```
// 分页和超时: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「分页和超时」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

分页和超时 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「分页和超时」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「分页和超时」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「分页和超时」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「分页和超时」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- 分页和超时 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 分页和超时 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「分页和超时」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

结果解释 核心讲解

本质：结果解释 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：结果解释 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

结果解释 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



结果解释 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

结果解释 工程示例

1. 最小工程骨架

```
// 结果解释: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「结果解释」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

结果解释 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「结果解释」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「结果解释」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「结果解释」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「结果解释」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- 结果解释 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 结果解释 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「结果解释」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

敏感数据脱敏 核心讲解

本质：敏感数据脱敏 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位：敏感数据脱敏 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

敏感数据脱敏 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



敏感数据脱敏 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

敏感数据脱敏 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

敏感数据脱敏 工程示例

1. 最小工程骨架

```
// 敏感数据脱敏: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「敏感数据脱敏」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

敏感数据脱敏 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「敏感数据脱敏」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「敏感数据脱敏」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「敏感数据脱敏」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「敏感数据脱敏」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- 敏感数据脱敏 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 敏感数据脱敏 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「敏感数据脱敏」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

NL2SQL 评估 核心讲解

本质： NL2SQL 评估 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。

定位： NL2SQL 评估 属于「数据库问答」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

NL2SQL 评估 是高价值也高风险的 AI 能力。它不是把自然语言直接翻译成 SQL，而是通过语义层、Schema Linking、SQL Plan、SQL Guard 和只读执行，把问题约束成安全查询。

1. 运行机制拆解

1. 第一步应识别业务指标，而不是猜表名。用户说“活跃员工”时，系统要知道指标口径、过滤条件和数据权限。
2. Schema Linking 把自然语言中的对象、指标、时间、维度映射到白名单表字段。没有语义层，模型会凭语感猜字段。
3. SQL Guard 要检查只读、LIMIT、超时、禁止关键字、schema 白名单、敏感字段和查询成本。
4. 结果解释要区分“SQL 查询结果”和“业务解释”。模型可以总结，但不能编造数据库没有返回的事实。

2. Java 工程落地抓手

- 准备 MetricDictionary、TableWhitelist、ColumnPolicy、SqlGuard、ReadOnlyDataSource 五个基础组件。
- 生成 SQL 前先生成查询计划，计划被校验后再渲染 SQL，减少模型自由发挥空间。
- 线上每条 SQL 都记录自然语言、计划、SQL、执行耗时、返回行数、脱敏字段和用户权限。

3. 核心架构图



NL2SQL 评估 组件拆解

1. 组件职责

1. 语义层：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
2. Schema Linking：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. SQL Plan：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
4. SQL Guard：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。
5. 只读执行：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 结果解释：负责把自然语言问题约束成安全、可解释、可审计的数据查询。不要让模型自由猜表、猜字段或直接执行高权限 SQL。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

NL2SQL 评估 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

NL2SQL 评估 工程示例

1. 最小工程骨架

```
// NL2SQL评估: secure NL2SQL
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.create(intent, allowedMetrics);
String sql = sqlGenerator.generate(plan);
sqlGuard.assertSelectOnly(sql).assertLimit().assertWhitelist();
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「NL2SQL 评估」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

NL2SQL 评估 生产坑与验收

1. 常见生产坑

1. 高权限账号执行 SQL：对「NL2SQL 评估」来说，处理建议是：数据库账号只读且限定 schema，SQL Guard 拦截危险语句。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有表字段白名单：对「NL2SQL 评估」来说，处理建议是：维护业务指标字典和表字段白名单，让模型先生成查询计划，再绑定真实表字段。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有 LIMIT 和超时：对「NL2SQL 评估」来说，处理建议是：所有 SQL 自动加 LIMIT、超时、分页和执行计划检查。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 敏感字段泄露：对「NL2SQL 评估」来说，处理建议是：敏感字段按角色脱敏或禁止返回，结果解释不泄露原始隐私明细。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- SQL 正确率
- 安全拦截率
- 查询成功率
- 结果解释准确率
- 敏感字段泄露率

3. 学完必须能回答

- NL2SQL 评估 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 NL2SQL 评估 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「NL2SQL 评估」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Agent workflows 模块架构图

Agent 是模型、工具、记忆、状态、规划、终止条件和护栏组成的受控执行系统。

这一章先看什么：Agent workflows 不是背概念，而是看它在系统里怎么落位。先把 没有最大步数；工具权限过大 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

Agent 本质 核心讲解

本质：Agent 本质 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：Agent 本质 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Agent 本质 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



Agent 本质 组件拆解

1. 组件职责

1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. Agent Runtime：这是「Agent 本质」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
3. Skills：这是「Agent 本质」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. Memory/State：负责让「Agent 本质」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Agent 本质 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Agent 本质 工程示例

1. 最小工程骨架

```
// Agent本质: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Agent 本质」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Agent 本质 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「Agent 本质」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「Agent 本质」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「Agent 本质」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「Agent 本质」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- Agent 本质 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Agent 本质 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Agent 本质」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

ReAct 模式 核心讲解

本质： ReAct 模式 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位： ReAct 模式 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

ReAct 模式 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「ReAct 模式」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

ReAct 模式 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「ReAct 模式」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「ReAct 模式」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「ReAct 模式」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

ReAct 模式 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

ReAct 模式 工程示例

1. 最小工程骨架

```
// ReAct模式: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「ReAct 模式」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

ReAct 模式 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「ReAct 模式」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「ReAct 模式」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「ReAct 模式」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「ReAct 模式」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- ReAct 模式的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 ReAct 模式失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「ReAct 模式」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Planning 规划 核心讲解

本质：Planning 规划 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：Planning 规划 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Planning 规划 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「Planning 规划」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Planning 规划 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「Planning 规划」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「Planning 规划」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「Planning 规划」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Planning 规划 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Planning 规划 工程示例

1. 最小工程骨架

```
// Planning规划: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Planning 规划」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Planning 规划 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「Planning 规划」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「Planning 规划」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「Planning 规划」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「Planning 规划」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- Planning 规划 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Planning 规划 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Planning 规划」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Memory 记忆 核心讲解

本质：Memory 记忆 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：Memory 记忆 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Memory 记忆 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「Memory 记忆」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Memory 记忆 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「Memory 记忆」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「Memory 记忆」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「Memory 记忆」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Memory 记忆 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Memory 记忆 工程示例

1. 最小工程骨架

```
// Memory记忆: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Memory 记忆」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Memory 记忆 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「Memory 记忆」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「Memory 记忆」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「Memory 记忆」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「Memory 记忆」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- Memory 记忆 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Memory 记忆 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Memory 记忆」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

State 状态 核心讲解

本质：State 状态 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：State 状态 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

State 状态 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「State 状态」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

State 状态 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「State 状态」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「State 状态」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「State 状态」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

State 状态 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

State 状态 工程示例

1. 最小工程骨架

```
// State状态: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「State 状态」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

State 状态 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「State 状态」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「State 状态」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「State 状态」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「State 状态」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- State 状态 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 State 状态 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「State 状态」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

ToolNode 核心讲解

本质：ToolNode 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：ToolNode 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

ToolNode 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「ToolNode」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

ToolNode 组件拆解

1. 组件职责

1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. Agent Runtime：这是「ToolNode」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
3. Skills：这是「ToolNode」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. Memory/State：负责让「ToolNode」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

ToolNode 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线: 主流程 虚线: 数据/工具补充 红线含义: 失败后循环或转人工

ToolNode 工程示例

1. 最小工程骨架

```
// ToolNode: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「ToolNode」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

ToolNode 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「ToolNode」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「ToolNode」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「ToolNode」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「ToolNode」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- ToolNode 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 ToolNode 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「ToolNode」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Multi-Agent 协作 核心讲解

本质：Multi-Agent 协作 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：Multi-Agent 协作 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Multi-Agent 协作 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「Multi-Agent 协作」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Multi-Agent 协作 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「Multi-Agent 协作」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「Multi-Agent 协作」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「Multi-Agent 协作」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Multi-Agent 协作 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Multi-Agent 协作 工程示例

1. 最小工程骨架

```
// Multi-Agent协作: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Multi-Agent 协作」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Multi-Agent 协作 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「Multi-Agent 协作」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「Multi-Agent 协作」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「Multi-Agent 协作」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「Multi-Agent 协作」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- Multi-Agent 协作 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Multi-Agent 协作 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Multi-Agent 协作」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

终止条件 核心讲解

本质：终止条件是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：终止条件属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

终止条件不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「终止条件」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

终止条件 组件拆解

1. 组件职责

1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. Agent Runtime：这是「终止条件」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
3. Skills：这是「终止条件」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. Memory/State：负责让「终止条件」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

终止条件 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

终止条件 工程示例

1. 最小工程骨架

```
// 终止条件: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「终止条件」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

终止条件 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「终止条件」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「终止条件」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「终止条件」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「终止条件」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- 终止条件 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 终止条件 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「终止条件」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Agent 安全 核心讲解

本质：Agent 安全 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：Agent 安全 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Agent 安全 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「Agent 安全」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Agent 安全 组件拆解

1. 组件职责

1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
2. Agent Runtime：这是「Agent 安全」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
3. Skills：这是「Agent 安全」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. Memory/State：负责让「Agent 安全」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Agent 安全 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Agent 安全 工程示例

1. 最小工程骨架

```
// Agent安全: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR：制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服：意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网：制度解读、流程办理、数据问答、文档生成。
- 运维/研发：日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Agent 安全」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

Agent 安全 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「Agent 安全」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「Agent 安全」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「Agent 安全」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「Agent 安全」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- Agent 安全 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Agent 安全 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Agent 安全」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

企业 Agent 流程 核心讲解

本质：企业 Agent 流程 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。

定位：企业 Agent 流程 属于「Agent 工作流」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

企业 Agent 流程 不是更长的 Prompt，也不是会连续聊天的机器人。Agent 是一个受控运行时：它围绕目标维护状态，选择技能和工具，执行观察，再根据结果决定继续、停止或转人工。

1. 运行机制拆解

1. Agent Runtime 负责接收目标、维护 State、选择下一步、调度 Skill/Tool、合并 observation、检查终止条件。
2. LLM 在 Agent 中像“大脑”，负责理解、规划和参数生成；但它不应该拥有权限、事务、数据库连接或外部动作执行权。
3. Memory 和 State 不一样。Memory 偏长期经验和用户偏好，State 是当前任务运行过程中的完整上下文、计划、工具结果和终止原因。
4. ReAct 适合工具问答和探索任务；固定 Workflow 适合企业流程；Multi-Agent 适合职责复杂但协同成本也更高的场景。

2. Java 工程落地抓手

- 先做 Workflow Agent，再做 ReAct Agent。生产项目不要一开始追无限自主循环。
- 为 Agent 设置 maxSteps、maxToolCalls、timeout、riskPolicy、humanApprovalPolicy 和 replayLog。
- 所有中间状态必须可回放，方便排查为什么调用了某个工具、为什么终止、为什么转人工。

3. 核心架构图



关键误区：Agent 最容易翻车在“无限循环”和“权限过大”。只要没有最大步数、工具分级、人工确认和状态回放，就不该上线。

判断是否学会：你能讲清「企业 Agent 流程」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

企业 Agent 流程 组件拆解

1. 组件职责

- 1. 大模型 LLM：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 2. Agent Runtime：这是「企业 Agent 流程」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 3. Skills：这是「企业 Agent 流程」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 4. MCP：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. Tools：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. Memory/State：负责让「企业 Agent 流程」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

企业 Agent 流程 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

企业 Agent 流程 工程示例

1. 最小工程骨架

```
// 企业Agent流程: controlled agent loop
while (!state.done() && state.steps() < MAX_STEPS) {
    Decision decision = llm.plan(state, allowedTools);
    Observation obs = toolNode.executeIfAllowed(decision);
    state = reducer.merge(state, decision, obs);
}
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「企业 Agent 流程」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

企业 Agent 流程 生产坑与验收

1. 常见生产坑

1. 没有最大步数：对「企业 Agent 流程」来说，处理建议是：设置最大步数、最大工具调用次数和最大耗时，超过阈值停止并转人工。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 工具权限过大：对「企业 Agent 流程」来说，处理建议是：工具按风险分级，只读工具可自动执行，写操作、外部通知、金额和审批必须确认。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 状态不可回放：对「企业 Agent 流程」来说，处理建议是：Agent State 保存用户目标、计划、工具调用、观察结果、中间答案和终止原因。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 高风险动作自动执行：对「企业 Agent 流程」来说，处理建议是：先用固定 Workflow Agent 上线，单链路稳定后再考虑 ReAct 或 Multi-Agent。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 任务完成率
- 平均步数
- 工具调用准确率
- 转人工率
- 高风险拦截率

3. 学完必须能回答

- 企业 Agent 流程 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 企业 Agent 流程 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「企业 Agent 流程」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

工程化与安全治理 模块架构图

把 AI 应用从能跑变成可上线、可追踪、可回滚、可持续优化。

这一章先看什么：工程化与治理 不是背概念，而是看它在系统里怎么落位。先把 没有日志；没有成本预算 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

模型网关 核心讲解

本质：模型网关 是企业级 AI 治理能力，决定系统能否真正上线。

定位：模型网关 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

模型网关 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



模型网关 组件拆解

1. 组件职责

1. 模型网关：这是「模型网关」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「模型网关」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「模型网关」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「模型网关」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

模型网关 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

模型网关 工程示例

1. 最小工程骨架

```
// 模型网关: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「模型网关」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

模型网关 生产坑与验收

1. 常见生产坑

1. 没有日志：对「模型网关」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「模型网关」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「模型网关」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「模型网关」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 模型网关 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 模型网关 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「模型网关」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt 管理 核心讲解

本质： Prompt 管理 是企业级 AI 治理能力，决定系统能否真正上线。

定位： Prompt 管理 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt 管理 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



Prompt 管理 组件拆解

1. 组件职责

1. 模型网关：这是「Prompt 管理」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「Prompt 管理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「Prompt 管理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「Prompt 管理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt 管理 工程示例

1. 最小工程骨架

```
// Prompt管理: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt 管理」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Prompt 管理 生产坑与验收

1. 常见生产坑

1. 没有日志：对「Prompt 管理」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「Prompt 管理」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「Prompt 管理」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「Prompt 管理」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- Prompt 管理的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt 管理失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt 管理」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

调用审计 核心讲解

本质：调用审计 是企业级 AI 治理能力，决定系统能否真正上线。

定位：调用审计 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

调用审计 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



调用审计 组件拆解

1. 组件职责

- 1. 模型网关：这是「调用审计」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 3. 权限策略：负责让「调用审计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 质量评估：负责让「调用审计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 6. 成本监控：负责让「调用审计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

调用审计 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

调用审计 工程示例

1. 最小工程骨架

```
// 调用审计: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「调用审计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

调用审计 生产坑与验收

1. 常见生产坑

1. 没有日志：对「调用审计」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「调用审计」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「调用审计」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「调用审计」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 调用审计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 调用审计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「调用审计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

AI 可观测性 核心讲解

本质：AI 可观测性 是企业级 AI 治理能力，决定系统能否真正上线。

定位：AI 可观测性 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

AI 可观测性 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



AI 可观测性 组件拆解

1. 组件职责

1. 模型网关：这是「AI 可观测性」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「AI 可观测性」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「AI 可观测性」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「AI 可观测性」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

AI 可观测性 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

AI 可观测性 工程示例

1. 最小工程骨架

```
// AI可观测性: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「AI 可观测性」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

AI 可观测性 生产坑与验收

1. 常见生产坑

1. 没有日志：对「AI 可观测性」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「AI 可观测性」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「AI 可观测性」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「AI 可观测性」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- AI 可观测性 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 AI 可观测性 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「AI 可观测性」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

质量评估 核心讲解

本质：质量评估 是企业级 AI 治理能力，决定系统能否真正上线。

定位：质量评估 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

质量评估 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



质量评估 组件拆解

1. 组件职责

1. 模型网关：这是「质量评估」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「质量评估」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「质量评估」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「质量评估」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

质量评估 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

质量评估 工程示例

1. 最小工程骨架

```
// 质量评估: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「质量评估」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

质量评估 生产坑与验收

1. 常见生产坑

1. 没有日志：对「质量评估」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「质量评估」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「质量评估」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「质量评估」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 质量评估 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 质量评估 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「质量评估」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

成本治理 核心讲解

本质：成本治理 是企业级 AI 治理能力，决定系统能否真正上线。

定位：成本治理 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

成本治理 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



成本治理 组件拆解

1. 组件职责

1. 模型网关：这是「成本治理」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「成本治理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「成本治理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「成本治理」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

成本治理 工程示例

1. 最小工程骨架

```
// 成本治理: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「成本治理」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

成本治理 生产坑与验收

1. 常见生产坑

1. 没有日志：对「成本治理」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「成本治理」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「成本治理」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「成本治理」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 成本治理 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 成本治理 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「成本治理」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

权限与租户 核心讲解

本质：权限与租户 是企业级 AI 治理能力，决定系统能否真正上线。

定位：权限与租户 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

权限与租户 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



权限与租户 组件拆解

1. 组件职责

- 1. 模型网关：这是「权限与租户」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 3. 权限策略：负责让「权限与租户」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 质量评估：负责让「权限与租户」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 6. 成本监控：负责让「权限与租户」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

权限与租户 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

权限与租户 工程示例

1. 最小工程骨架

```
// 权限与租户: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「权限与租户」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

权限与租户 生产坑与验收

1. 常见生产坑

1. 没有日志：对「权限与租户」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「权限与租户」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「权限与租户」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「权限与租户」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 权限与租户 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 权限与租户 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「权限与租户」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

敏感信息脱敏 核心讲解

本质：敏感信息脱敏 是企业级 AI 治理能力，决定系统能否真正上线。

定位：敏感信息脱敏 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

敏感信息脱敏 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



敏感信息脱敏 组件拆解

1. 组件职责

- 1. 模型网关：这是「敏感信息脱敏」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 3. 权限策略：负责让「敏感信息脱敏」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 质量评估：负责让「敏感信息脱敏」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 6. 成本监控：负责让「敏感信息脱敏」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

敏感信息脱敏 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

敏感信息脱敏 工程示例

1. 最小工程骨架

```
// 敏感信息脱敏: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「敏感信息脱敏」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

敏感信息脱敏 生产坑与验收

1. 常见生产坑

1. 没有日志：对「敏感信息脱敏」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「敏感信息脱敏」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「敏感信息脱敏」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「敏感信息脱敏」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 敏感信息脱敏 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 敏感信息脱敏 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「敏感信息脱敏」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

Prompt 注入防护 核心讲解

本质： Prompt 注入防护 是企业级 AI 治理能力，决定系统能否真正上线。

定位： Prompt 注入防护 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

Prompt 注入防护 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



关键误区： 只靠 Prompt 做安全是脆弱的。真正的安全来自权限系统、数据边界、工具边界、输出校验和审计回放。

判断是否学会： 你能讲清「Prompt 注入防护」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

Prompt 注入防护 组件拆解

1. 组件职责

1. 模型网关：这是「Prompt 注入防护」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
3. 权限策略：负责让「Prompt 注入防护」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
5. 质量评估：负责让「Prompt 注入防护」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
6. 成本监控：负责让「Prompt 注入防护」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

Prompt 注入防护 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

Prompt 注入防护 工程示例

1. 最小工程骨架

```
// Prompt注入防护: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「Prompt 注入防护」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

Prompt 注入防护 生产坑与验收

1. 常见生产坑

1. 没有日志：对「Prompt 注入防护」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「Prompt 注入防护」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「Prompt 注入防护」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「Prompt 注入防护」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- Prompt 注入防护 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 Prompt 注入防护 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「Prompt 注入防护」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

灰度与回滚 核心讲解

本质：灰度与回滚 是企业级 AI 治理能力，决定系统能否真正上线。

定位：灰度与回滚 属于「工程化与治理」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

灰度与回滚 决定 AI 应用能不能真正上线。治理不是最后补文档，而是从第一天就嵌入模型调用、Prompt、RAG、工具、Agent 和发布流程。

1. 运行机制拆解

1. 模型网关治理调用入口：统一路由、限流、预算、日志、超时、fallback 和 provider 差异。
2. 质量治理依赖评估集：没有固定样本，就无法判断 Prompt、模型、RAG 策略和工具描述是否变好。
3. 安全治理要分层：输入隔离、知识权限、工具权限、输出校验、敏感数据脱敏和高风险人工确认。
4. 发布治理要可回滚：模型、Prompt、检索策略、工具 schema 都要版本化和灰度，不要一次性全量替换。

2. Java 工程落地抓手

- 建立 AiAuditLog、PromptRegistry、EvaluationSet、CostBudget、PolicyEngine、ReleaseStrategy 这些基础设施。
- 把质量、成本、延迟、安全拦截、拒答、人工采纳率做成仪表盘，别等用户投诉才发现问题。
- 把失败样本回流成评估集，而不是只在群里讨论“这次模型又答错了”。

3. 核心架构图



灰度与回滚 组件拆解

1. 组件职责

- 1. 模型网关：这是「灰度与回滚」链路中的一个职责点。要讲清它的输入、输出、上下游依赖、失败表现，以及应该由模型、Java 服务、数据库还是人工来兜底。
- 2. Prompt 管理：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 3. 权限策略：负责让「灰度与回滚」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 4. 审计日志：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 5. 质量评估：负责让「灰度与回滚」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。
- 6. 成本监控：负责让「灰度与回滚」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

灰度与回滚 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

灰度与回滚 工程示例

1. 最小工程骨架

```
// 灰度与回滚: governance wrapper
Policy policy = policyEngine.check(user, task, dataScope);
String requestId = audit.start(user, model, promptVersion);
AiResponse response = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(response, rules);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「灰度与回滚」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

灰度与回滚 生产坑与验收

1. 常见生产坑

1. 没有日志：对「灰度与回滚」来说，处理建议是：AI 调用日志记录 requestId、用户、模型、Prompt 版本、知识来源、工具调用、token、耗时和结果。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 没有成本预算：对「灰度与回滚」来说，处理建议是：设置团队、业务线、用户和模型维度预算，超预算时降级模型或压缩上下文。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有灰度和回滚：对「灰度与回滚」来说，处理建议是：模型、Prompt、RAG 策略发布都走灰度，对照评估集胜出后再扩大流量。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 只靠 Prompt 防注入：对「灰度与回滚」来说，处理建议是：把注入防护放在输入隔离、知识过滤、工具权限、输出校验四层。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 审计完整率
- 单位任务成本
- 质量评分
- 安全拦截率
- 灰度胜率

3. 学完必须能回答

- 灰度与回滚 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 灰度与回滚 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「灰度与回滚」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

综合设计与架构表达 模块架构图

训练把业务目标拆成 AI 能力、数据流、控制流、风险点和验收指标的能力。

这一章先看什么： AI 应用架构 不是背概念，而是看它在系统里怎么落位。先把 先选模型再找场景；只画组件不画数据流 这类问题看懂，再回头看图，才知道每条线为什么这么连。



主线：业务请求到结果返回 补充：知识/工具/数据调用 治理：权限、审计、评估、反馈

从业务价值出发 核心讲解

本质：从业务价值出发 是 AI 项目从想法到落地的架构表达能力。

定位：从业务价值出发 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

从业务价值出发 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



从业务价值出发 组件拆解

1. 组件职责

1. 业务价值：回答“为什么要做”。学习「从业务价值出发」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
6. 治理闭环：负责让「从业务价值出发」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

从业务价值出发 工程示例

1. 最小工程骨架

```
// 从业务价值出发: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「从业务价值出发」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

从业务价值出发 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「从业务价值出发」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「从业务价值出发」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「从业务价值出发」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「从业务价值出发」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 从业务价值出发 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 从业务价值出发 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「从业务价值出发」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

数据流设计 核心讲解

本质：数据流设计 是 AI 项目从想法到落地的架构表达能力。

定位：数据流设计 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

数据流设计 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



关键误区：最常见的误区是把 AI 架构画成“前端 -> 模型 -> 答案”。真实企业链路至少还要有知识、工具、权限、审计、评估、降级和反馈，否则 Demo 越漂亮，上线越危险。

判断是否学会：你能讲清「数据流设计」的输入、输出、边界、失败表现和验收指标，并能把上面的图复画出来。

数据流设计 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「数据流设计」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「数据流设计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

数据流设计 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

数据流设计 工程示例

1. 最小工程骨架

```
// 数据流设计: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「数据流设计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

数据流设计 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「数据流设计」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「数据流设计」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「数据流设计」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「数据流设计」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 数据流设计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 数据流设计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「数据流设计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

控制流设计 核心讲解

本质：控制流设计 是 AI 项目从想法到落地的架构表达能力。

定位：控制流设计 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

控制流设计 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



控制流设计 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「控制流设计」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「控制流设计」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

控制流设计 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

控制流设计 工程示例

1. 最小工程骨架

```
// 控制流设计: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「控制流设计」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

控制流设计 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「控制流设计」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「控制流设计」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「控制流设计」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「控制流设计」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 控制流设计 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 控制流设计 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「控制流设计」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

风险点标注 核心讲解

本质：风险点标注 是 AI 项目从想法到落地的架构表达能力。

定位：风险点标注 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

风险点标注 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



风险点标注 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「风险点标注」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「风险点标注」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

风险点标注 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

风险点标注 工程示例

1. 最小工程骨架

```
// 风险点标注: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「风险点标注」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

风险点标注 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「风险点标注」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「风险点标注」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「风险点标注」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「风险点标注」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 风险点标注 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 风险点标注 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「风险点标注」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

验收指标 核心讲解

本质：验收指标 是 AI 项目从想法到落地的架构表达能力。

定位：验收指标 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

验收指标 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。

判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



验收指标 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「验收指标」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「验收指标」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

验收指标 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

验收指标 工程示例

1. 最小工程骨架

```
// 验收指标: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「验收指标」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

验收指标 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「验收指标」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「验收指标」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「验收指标」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「验收指标」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 验收指标 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 验收指标 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「验收指标」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

分阶段路线 核心讲解

本质：分阶段路线 是 AI 项目从想法到落地的架构表达能力。

定位：分阶段路线 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

分阶段路线 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



分阶段路线 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「分阶段路线」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「分阶段路线」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

分阶段路线 工程示例

1. 最小工程骨架

```
// 分阶段路线: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「分阶段路线」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

分阶段路线 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「分阶段路线」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「分阶段路线」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「分阶段路线」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「分阶段路线」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 分阶段路线 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 分阶段路线 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「分阶段路线」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

团队角色 核心讲解

本质：团队角色 是 AI 项目从想法到落地的架构表达能力。

定位：团队角色 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

团队角色 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



团队角色 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「团队角色」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「团队角色」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

团队角色 工程示例

1. 最小工程骨架

```
// 团队角色: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「团队角色」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

团队角色 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「团队角色」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「团队角色」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「团队角色」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「团队角色」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 团队角色 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 团队角色 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「团队角色」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

架构图表达 核心讲解

本质：架构图表达 是 AI 项目从想法到落地的架构表达能力。

定位：架构图表达 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

架构图表达 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



架构图表达 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「架构图表达」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「架构图表达」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

架构图表达 全流程图

复杂知识点必须像流程图一样拆清楚：谁发起、谁理解、谁调用、谁执行、谁校验、谁反馈。下面这张图用于建立全局视角。



实线：主流程 虚线：数据/工具补充 红线含义：失败后循环或转人工

架构图表达 工程示例

1. 最小工程骨架

```
// 架构图表达: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「架构图表达」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

架构图表达 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「架构图表达」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「架构图表达」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「架构图表达」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「架构图表达」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 架构图表达 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 架构图表达 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「架构图表达」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

汇报方式 核心讲解

本质：汇报方式 是 AI 项目从想法到落地的架构表达能力。

定位：汇报方式 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

汇报方式 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



汇报方式 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「汇报方式」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「汇报方式」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

汇报方式 工程示例

1. 最小工程骨架

```
// 汇报方式: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「汇报方式」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像，要看是否可验证、可追踪、可回滚。

汇报方式 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「汇报方式」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「汇报方式」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「汇报方式」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「汇报方式」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 汇报方式 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 汇报方式 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「汇报方式」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

后续实战准备 核心讲解

本质：后续实战准备 是 AI 项目从想法到落地的架构表达能力。

定位：后续实战准备 属于「AI 应用架构」能力。不要先背名词，先判断它在系统里解决什么问题、依赖哪些组件、失败时由谁兜底。

先给结论

后续实战准备 要解决的不是“会不会接模型”，而是把一个概率生成能力放进确定性的企业系统里。判断一个 AI 架构是否成熟，关键看它有没有把目标、上下文、知识、工具、执行、审计和反馈串成闭环。

1. 运行机制拆解

1. 第一步不是选模型，而是拆业务目标：这个功能到底提高效率、降低风险、改善质量，还是只是看起来新鲜。目标越虚，后面的 Prompt、RAG、Agent 都会变成堆概念。
2. 第二步拆能力边界：模型负责理解和生成，知识库负责事实补充，工具负责实时数据和业务动作，Java 后端负责权限、事务、规则、审计和兜底。
3. 第三步画数据流和控制流：用户输入如何进入上下文，知识片段如何被选择，工具参数如何被校验，结果如何被人工确认或自动进入下一步。
4. 第四步设计失败路径：模型不确定、资料不足、工具失败、权限不足、成本超限、用户不满意时，系统应该拒答、降级、转人工还是重新检索。

2. Java 工程落地抓手

- Java 项目里先抽出 AiApplicationService 或 AiGateway，Controller 只处理 HTTP，不要把模型调用、Prompt、RAG 和工具执行混在控制层。
- 把每条 AI 链路都当成业务流程建模：Request、Context、Decision、Evidence、Action、AuditRecord 这些对象要清楚。
- 验收时不要只看“回答像不像”，还要看可追溯来源、权限正确性、失败可解释、成本可控和人工采纳率。

3. 核心架构图



后续实战准备 组件拆解

1. 组件职责

- 1. 业务价值：回答“为什么要做”。学习「后续实战准备」时，先判断它是否能带来效率、质量、成本、风险或体验改进，否则很容易变成模型能力展示。
- 2. 模型能力：负责理解、生成、归纳、规划或总结，但不负责事实真实性、最终权限和业务事务。它的输出必须被上下文、工具、检索和服务端校验约束。
- 3. 上下文工程：负责告诉模型“看什么、做什么、不能做什么、按什么格式输出”。它是模型行为的接口协议，必须可版本化、可评估、可回滚。
- 4. 私有知识：负责把企业事实变成可检索、可引用、可过滤的上下文。关键不是把资料塞给模型，而是保留来源、权限、版本、时间和召回证据。
- 5. 业务工具：负责连接外部系统。模型只能提出调用意图，Java 服务必须校验参数、权限、幂等、风险等级和审计日志。
- 6. 治理闭环：负责让「后续实战准备」从 Demo 变成可上线能力。它要记录过程、控制风险、评估质量、限制成本，并把失败样本回流到下一轮优化。

2. 组件协作



3. 边界提醒

- 模型负责理解、生成、规划或总结，不负责最终权限和真实业务执行。
- 数据进入模型前要筛选、脱敏、过滤权限和压缩上下文。
- 每个组件都要能测试、记录、替换和降级。
- 最终结果必须能被业务人员验证，而不是只让技术人员觉得炫。

后续实战准备 工程示例

1. 最小工程骨架

```
// 后续实战准备: architecture decision
Goal goal = clarifyBusinessValue(request);
CapabilityMap caps = splitModelKnowledgeTools(goal);
RiskMap risks = identifyRisks(caps);
AiFlow flow = designDataAndControlFlow(caps, risks);
Metrics metrics = defineAcceptance(flow);
```

2. 可迁移场景

- HR: 制度问答、候选人匹配、面试总结、招聘漏斗分析。
- 客服: 意图识别、知识库回答、工单摘要、工具查询。
- 政务/企业内网: 制度解读、流程办理、数据问答、文档生成。
- 运维/研发: 日志分析、故障排查、代码解释、规范问答。

3. 不要这样用

- 不要把「后续实战准备」写成孤立 Demo。
- 不要让模型绕过 Java 后端权限和审计。
- 不要只看回答像不像, 要看是否可验证、可追踪、可回滚。

后续实战准备 生产坑与验收

1. 常见生产坑

1. 先选模型再找场景：对「后续实战准备」来说，处理建议是：先写业务价值假设和验收指标，再决定是否需要模型、RAG、工具或 Agent。这类治理必须落到系统机制上，而不是停留在口头规范。
2. 只画组件不画数据流：对「后续实战准备」来说，处理建议是：补数据流图，标明用户输入、业务数据、检索数据、模型上下文、工具参数和审计日志流向。这类治理必须落到系统机制上，而不是停留在口头规范。
3. 没有失败路径：对「后续实战准备」来说，处理建议是：为模型失败、检索为空、权限不足、成本超限、人工拒绝分别设计分支。这类治理必须落到系统机制上，而不是停留在口头规范。
4. 没有验收指标：对「后续实战准备」来说，处理建议是：把准确率、节省时间、采纳率、成本、延迟、安全事件写进项目验收。这类治理必须落到系统机制上，而不是停留在口头规范。

2. 推荐验收指标

- 业务节省时间
- 回答准确率
- 人工采纳率
- 失败可解释率
- 审计完整率

3. 学完必须能回答

- 后续实战准备 的输入、输出、依赖组件分别是什么？
- 哪些能力交给模型，哪些必须由 Java 后端负责？
- 如果 后续实战准备 失败，是重试、降级、拒答、转人工，还是回滚配置？
- 如何记录日志、设计评估集、观察成本和质量？

小练习：拿你熟悉的一个业务系统，画出「后续实战准备」的数据流和控制流，并标出 3 个高风险点、3 个监控指标、1 个降级方案。

后续深度学习资料

- Spring AI Reference: <https://docs.spring.io/spring-ai/reference/>
- Spring AI Tool Calling: <https://docs.spring.io/spring-ai/reference/api/tools.html>
- Spring AI Advisors: <https://docs.spring.io/spring-ai/reference/api/advisors.html>
- Spring AI Examples: <https://github.com/spring-projects/spring-ai-examples>
- Spring AI Alibaba: <https://github.com/alibaba/spring-ai-alibaba>
- Model Context Protocol Architecture: <https://modelcontextprotocol.io/specification/2025-06-18/architecture>
- MCP Java SDK: <https://github.com/modelcontextprotocol/java-sdk>
- Microsoft Advanced RAG: <https://learn.microsoft.com/en-us/azure/developer/ai/advanced-retrieval-augmented-generation>
- Azure AI Search RAG: <https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview>
- LangChain4j Documentation: <https://docs.langchain4j.dev/>
- OpenAI Java SDK: <https://github.com/openai/openai-java>
- OpenSquilla: <https://github.com/opensquilla/opensquilla>
- Dify: <https://github.com/langgenius/dify>
- RAGFlow: <https://github.com/infiniflow/ragflow>